

**PROJECT TITLE : MULTI – LINE INSURANCE POLICY
AND CLAIMS MANAGEMENT SYSTEM**

**COLLEGE NAME : ARASU ENGINEERING COLLEGE,
KUMBAKONAM**

**TEAM LEADER : A.KANIKA
(WITH EMAIL ID) (kanikaannadurii@gmail.com)**

**TEAM MEMBER 1 : T.AADHIRAI
(WITH EMAIL ID) (aadhirait2005@ gmail.com)**

**TEAM MEMBER 2 : J.AYISHA
(WITH EMAIL ID) (jayasutha7u@gmail.com)**

**TEAM MEMBER 3 : R.ABINAYA
(WITH EMAIL ID) (rjabinaya08@gmail.com)**

**TEAM MEMBER 4 : S.INDIRA GANTHI
(WITH EMAIL ID) (kp191140@gmail.com)**

MULTI – LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEMS

User Story:

This project delivers a single Salesforce platform for handling Vehicle, Property and Life insurance policies together with their claims. Agents build and maintain policy records on a flexible data model that uses policy-specific Record Types and Field Sets. A guided Screen Flow walks the agent through quoting, while Apex logic works out the premium from the policy and vehicle details.

When a claim is logged, it is sent automatically to the right queue for its policy type. Claims above the high-value limit go through an approval chain that passes from a Senior Adjuster to a Claims Manager. Adjusters get a Lightning Web Component dashboard that lists only their own claims, showing amount, policy type, policy holder, status and days open. Together these pieces speed up policy handling, tighten claim processing, improve data accuracy, give better operational visibility and raise the quality of customer service.

1. Introduction

1.1 Project Overview

The Multi-Line Insurance Policy and Claims Management System is a Salesforce application that brings Vehicle, Property and Life insurance policies and claims onto one platform. It replaces scattered, manual working methods with structured custom objects, Record Types, Field Sets, Flows, Apex automation, Approval Processes and Lightning Web Components.

Agents create policies through a flexible data model, and a guided Screen Flow makes quoting simple. The system checks the entered data and uses Apex to calculate premiums from factors such as the policy state and the vehicle model year. Once a claim is created, automation reads the related policy type and places the claim in the matching claims queue.

Adjusters work from an LWC-based Claims Dashboard that shows claim number, policy type, policy holder, claim amount, approval status and days open. Claims above the set threshold follow an approval workflow through Senior Adjusters and Claims Managers. Sharing rules and role-based Permission Sets control who can see and do what.

1.2 Purpose

The purpose of the project is to give insurance teams one Salesforce-based place to manage Vehicle, Property and Life policies and claims, and to remove the slow, manual and inconsistent quoting, issuing and claim-handling practices of legacy processes.

- Standardise and speed up policy quoting and issuance.
- Start and route claims automatically, based on policy type and claim size.
- Give a 360-degree view of customer, policy and claim data.
- Support guided quoting, automated premium and claim processing, data validation and an adjuster dashboard.

In short, the project aims to improve efficiency, data accuracy, claims handling and customer experience across all insurance lines.

2. Ideation Phase

2.1 Problem Statements

The table below lists the ten customer problem statements that drove the project. The overview diagram that follows shows the same problems visually.



ID	Role	Trying to	But	Because	Feels
PS-1	Operations Manager	standardise policy quoting and issuance	the process is manual, slow and inconsistent	legacy systems and processes are fragmented	Frustrated, concerned
PS-2	Insurance Agent	create and manage policies efficiently	each insurance line needs different information	Vehicle, Property and Life policies have different data needs	Overwhelmed
PS-3	Insurance Agent	give accurate policy quotes	quoting takes many steps and manual work	there is no unified quoting platform	Frustrated, uncertain
PS-4	Claims Manager	manage claims efficiently	review and routing are mostly manual	claims handling is neither centralised nor automated	Stressed
PS-5	Claims Adjuster	work through my assigned claims	I have no focused view of my workload	claim data is not available in one place	Frustrated, less productive
PS-6	Claims Manager	send claims to the right team	claims must follow policy type and severity	routing is complex and needs automation	Worried about delays
PS-7	Senior Adjuster	process high-value claims efficiently	approval needs several review levels	high-value claims need structured approval and escalation	Pressured

ID	Role	Trying to	But	Because	Feels
PS-8	Operations Manager	keep policy data accurate	products need different fields and validations	the three policy types have varying data needs	Worried about data quality
PS-9	Customer	get my policy and claim handled quickly	manual processing causes delays	policy and claim processes are fragmented	Uncertain, dissatisfied
PS-10	Insurance Management	see customer, policy and claim data together	information is spread across systems	there is no unified platform	Worried about efficiency

2.2 Empathy Map Canvas

The persona chosen for the empathy map is the Insurance Claims Adjuster, because this role feels the effect of slow, manual claim handling most directly.

- **Think & Feel:** Worried about claim delays and the size of the workload.
- **Hear:** Customers and colleagues asking when claims will be resolved.
- **See:** Many claims, policy details and approval statuses spread across screens.
- **Say & Do:** Reviews claims and follows up on pending approvals.
- **Pain:** Manual processing, complicated routing and little visibility.
- **Gain:** A claims dashboard, automation and faster approvals.

Empathy Map & Day-in-the-Life - Claims Adjuster

Left: what the adjuster thinks / says / does / feels. Right: how the day feels before and after the system.

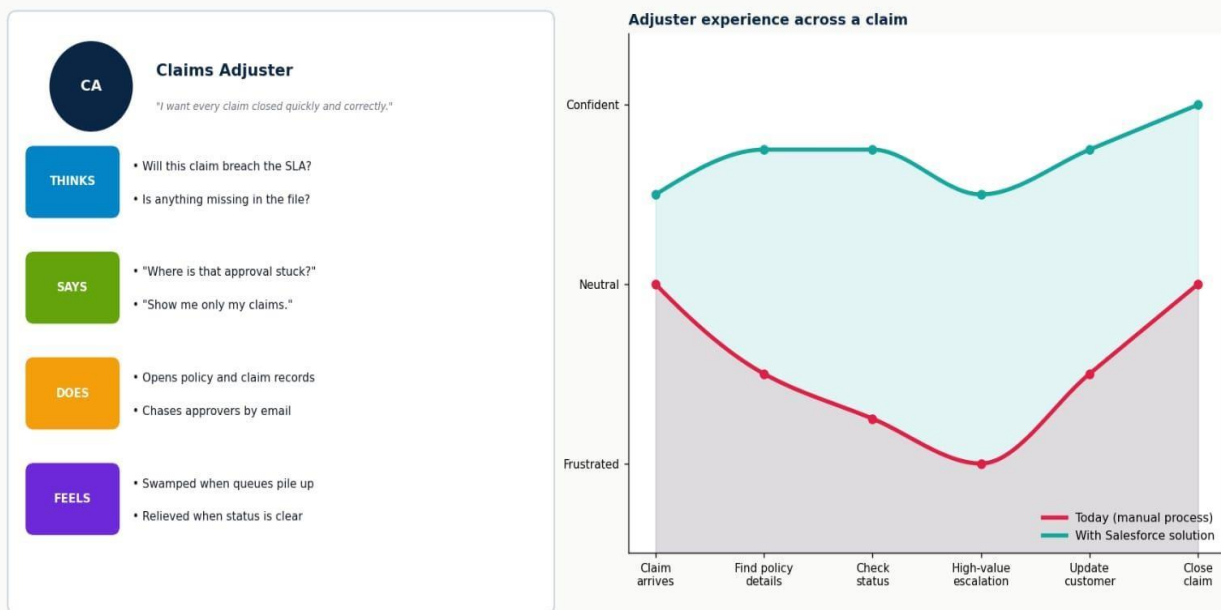


Figure 2: Empathy map for the Claims Adjuster persona

2.3 Brainstorming and Idea Prioritisation

Four approaches were considered for managing policies and claims:

- **Manual registers** – policy and claim data kept on paper or by hand.
- **Spreadsheet tracking** – policies, claims and approvals tracked in Excel files.
- **Standalone portal** – a separate custom portal for policies and claims.
- **Salesforce CRM** – one unified platform for policies, claims, automation and reporting.

Salesforce CRM was selected for these reasons:

- **Automation** – quoting, premium calculation, claim routing and approvals can all be automated.
- **Scalability** – it handles Vehicle, Property and Life insurance and can grow to more lines.
- **Real-time visibility** – customer, policy and claim data is visible in one place.
- **Unified platform** – policy and claim processes live in the same system.
- **Better efficiency** – less manual effort and faster claim handling.

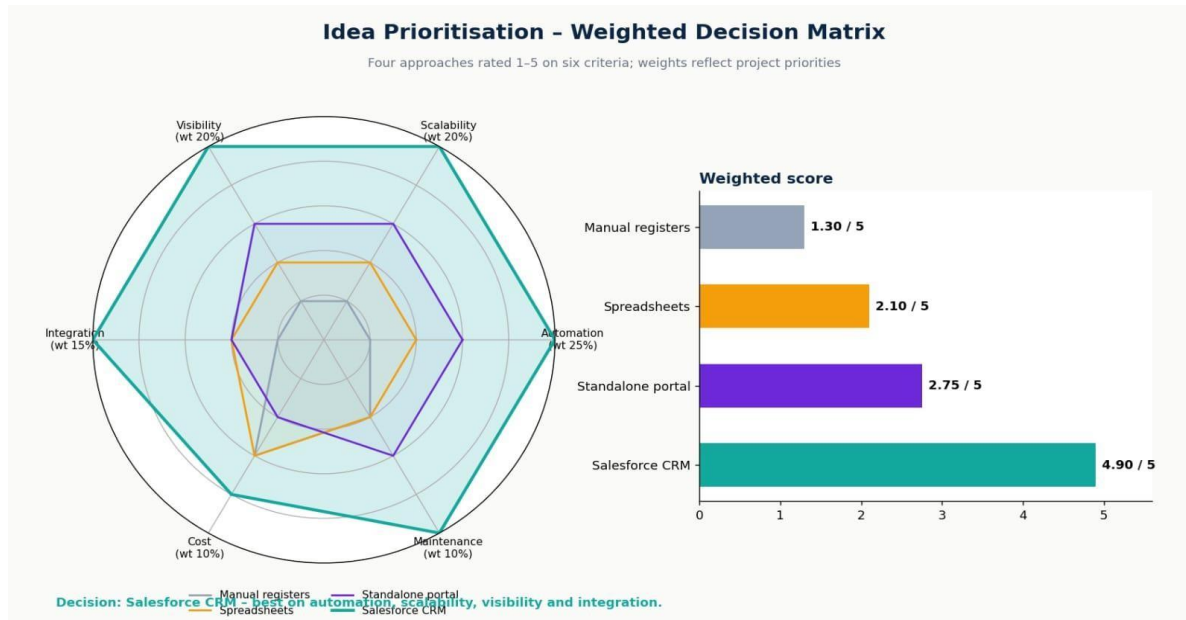


Figure 3: Brainstorming outcome and prioritisation scores

3. Requirement Analysis Phase

3.1 Customer Journey Map

The journey covers everything from the first insurance enquiry to reporting, in these stages:

- Insurance enquiry
- Policy selection (Vehicle / Property / Life)
- Quote request
- Capture of policy information
- Premium calculation
- Policy issuance
- Claim initiation
- Claim routing
- Claim review and approval
- Claim resolution
- Customer confirmation
- Reporting and visibility

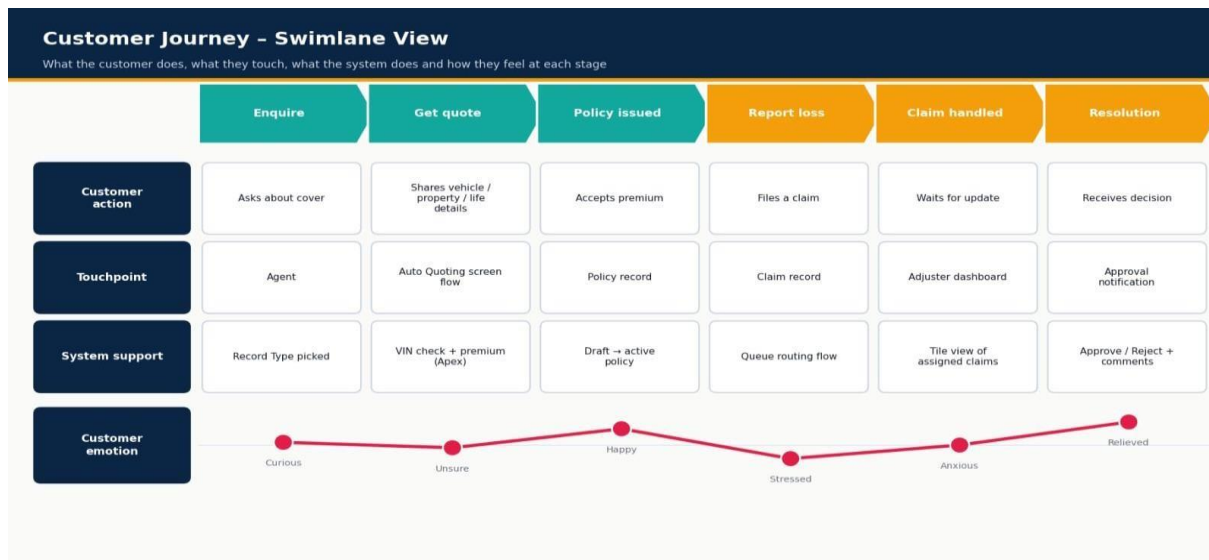


Figure 4: Customer journey map for the requirement analysis pha

3.2 Solution Requirements

Functional Requirements

ID	Requirement
FR-01	Users can create and manage Policy records for Vehicle, Property and Life insurance.
FR-02	Separate Record Types exist for Vehicle, Property and Life policies.
FR-03	Product-specific details are captured, such as VIN for Vehicle and Square Footage / Year Built for Property.
FR-04	A guided Auto Quoting Screen Flow collects policy holder, start date, state, VIN and model year.
FR-05	A draft policy is created while the quote is being prepared.
FR-06	Premiums are calculated by Apex-based logic.
FR-07	Vehicle VINs are validated and must be exactly 17 characters.
FR-08	Users can create and manage Claim records linked to policies.

ID	Requirement
FR-09	Claim Record Types exist for Accident, Property and Life claims.
FR-10	Claims are routed automatically to the correct queue based on the related policy type.
FR-11	High-value claims are identified using a configured amount threshold.
FR-12	Qualifying claims go through a multi-level approval involving Senior Adjusters and Claims Managers.
FR-13	Approvers can Approve or Reject a claim and add comments.
FR-14	Claims Adjusters get an LWC Claims Dashboard of their assigned claims.
FR-15	The dashboard shows Claim Number, Policy Type, Policy Holder, Claim Amount, Status and Days Open.
FR-16	The dashboard can be filtered by All, Auto, Property and Life.
FR-17	Related Policy and Claim records are updated automatically by Record-Triggered Flows.
FR-18	State-based access and sharing is supported for the relevant users and records.
FR-19	Permission Sets give role-specific access to Agents, Adjusters and Managers.
FR-20	Customer, Policy and Claim data can be reported on from the central platform.

Non-Functional Requirements

These describe the expected quality of the solution. Some are derived from the project objectives rather than written as formal NFRs.

ID	Area	Description
NFR-01	Usability	Guided Screen Flows keep clicks to a minimum and make quoting easy for agents.

ID	Area	Description
NFR-02	Performance	Quoting, premium calculation, claim routing and the dashboard should respond quickly for daily operations.
NFR-03	Scalability	The solution supports three insurance lines now and can take on more products later.
NFR-04	Data integrity	Policy and claim data is validated so it stays accurate and consistent.
NFR-05	Security	Access follows the user's responsibility, using state-based sharing and role-specific Permission Sets.
NFR-06	Maintainability	Modular components (Flows, Apex, LWC, Record Types, Field Sets) can be maintained independently.
NFR-07	Reliability	Claim routing, premium calculation and approvals run consistently according to the business rules.
NFR-08	Extensibility	Product-specific fields fit the data model without a separate structure for each line.
NFR-09	Visibility	Users can quickly reach customer, policy and claim data through records and the dashboard.
NFR-10	User adoption	An intuitive experience reduces manual work and helps agents and claims users adopt the system.

3.3 Data Flow Diagram

Overall flow: Customer / Agent → Screen Flow → Policy Validation → Policy__c → PremiumCalculator → Claim_c → Claim Routing Flow → Claims Queue → Claims Adjuster → Approval Process → Claim Update → LWC Dashboard / Reports.

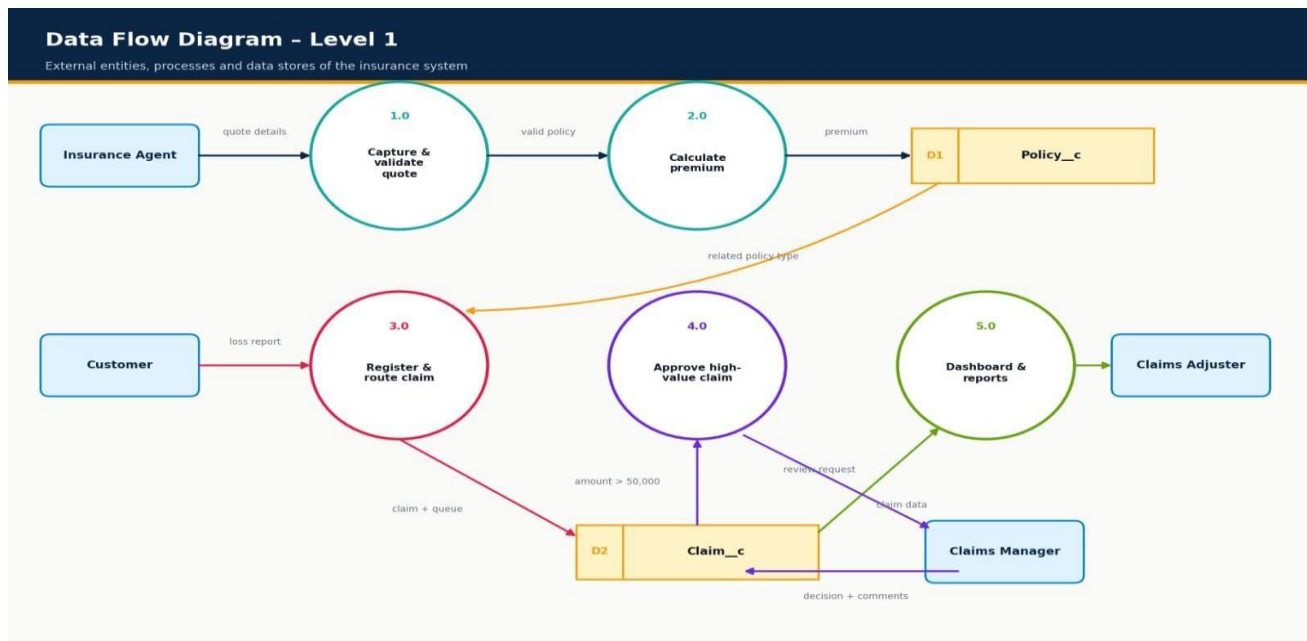


Figure 5: Data flow diagram – policy/quoting flow and claims flow

Step-by-step data flow

- **Step 1 – Input:** the agent receives the customer and policy information.
- **Step 2 – Capture:** the AutoQuoting Screen Flow collects policy holder, start date, state, VIN and model year.
- **Step 3 – Validation:** Salesforce checks the data, including the 17-character VIN rule for Auto policies.
- **Step 4 – Policy creation:** valid data is saved in Policy_c under the Vehicle, Property or Life Record Type.
- **Step 5 – Premium:** the policy is passed to the PremiumCalculator Apex class to get the premium.
- **Step 6 – Claim creation:** when a loss happens, a Claim_c record is created and linked to the policy.
- **Step 7 – Routing:** a Record-Triggered Flow finds the policy type and assigns the claim to the Auto, Property or Life queue.
- **Step 8 – Review:** the adjuster opens assigned claims on the Claims Dashboard and reviews them.
- **Step 9 – High-value approval:** claims above the threshold go to the Senior Adjuster and then the Claims Manager.

- **Step 10 – Decision:** the approver chooses Approve or Reject, adds comments, and the status is updated.
- **Step 11 – Reporting:** the dashboard shows amount, policy type, holder, status and days open.

3.4 Technology Stack

Layer	Technology
User interface	Salesforce Lightning, Lightning Web Components (LWC)
Logic	Salesforce Flows, Apex
Database	Custom objects Policy__c and Claim__c, plus Contact and User
Automation	Record-Triggered Flows, Validation Rules, Approval Processes
Processing	Apex Premium Calculator and Apex controllers
Data configuration	Record Types, Field Sets, custom fields
Security	Permission Sets and state-based Sharing Rules
Reporting	Salesforce Reports and Dashboards
Testing	Apex test classes

4. Project Design Phase

4.1 Problem–Solution Fit

Salesforce suits this project because each key challenge maps to a configurable feature: Screen Flows for quoting, Record Types and Field Sets for different products, Apex for premium logic, Record-Triggered Flows for routing, Approval Processes for high-value claims, LWC for adjuster visibility, Validation Rules for data quality, and Permission Sets with Sharing Rules for access control.

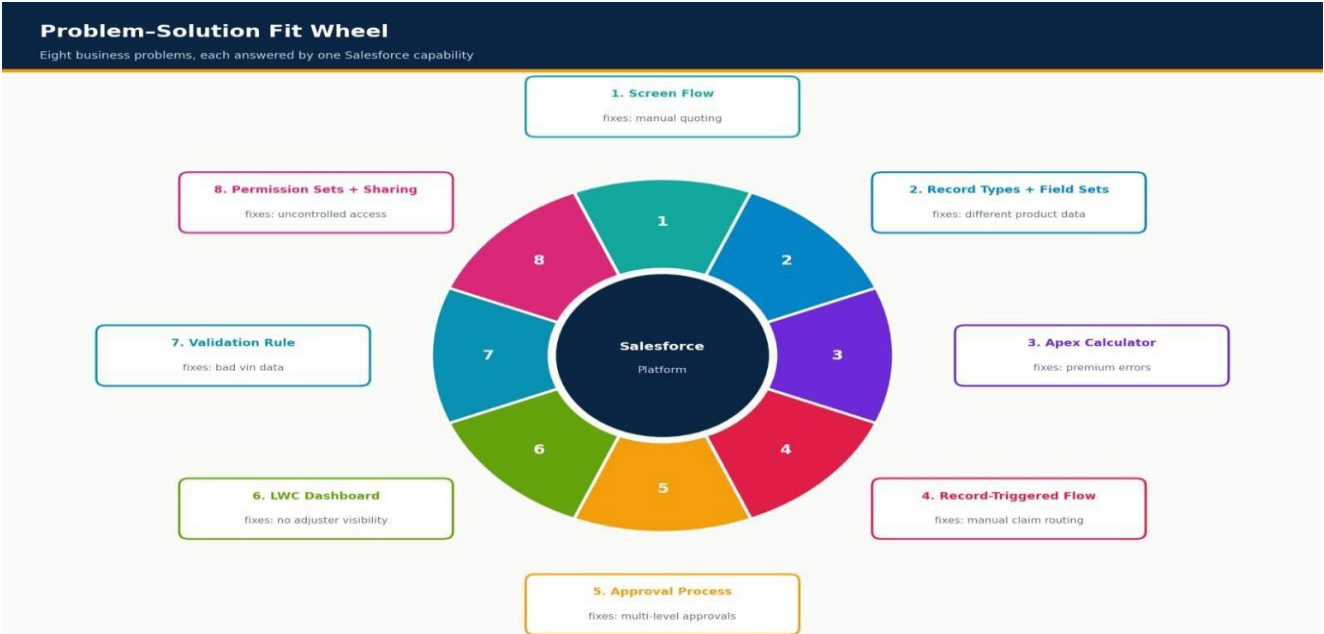


Figure 6: Problem–solution fit

4.2 Proposed Solution

Parameter	Description
Problem	Manual and inconsistent policy quoting, issuance and claims handling
Solution	A Salesforce CRM-based Insurance Policy and Claims Management System
Innovation	Automated quoting, premium calculation, claim routing and high-value claim approval
Impact	Faster policy processing, efficient claim handling and better customer service

Parameter	Description
Business model	Central management of Vehicle, Property and Life insurance
Scalability	A flexible data model that supports more insurance products in future
Visibility	Central customer, policy and claim data with a Claims Dashboard
Security	Role-based and state-based access through Permission Sets and Sharing Rules

This matches the project goal of one Salesforce platform that standardises policy processes, automates claims and gives a 360-degree view of customer, policy and claim data.

4.3 Solution Architecture

The solution uses a layered Salesforce architecture covering quoting, premium calculation, claims processing, approvals, security and reporting.

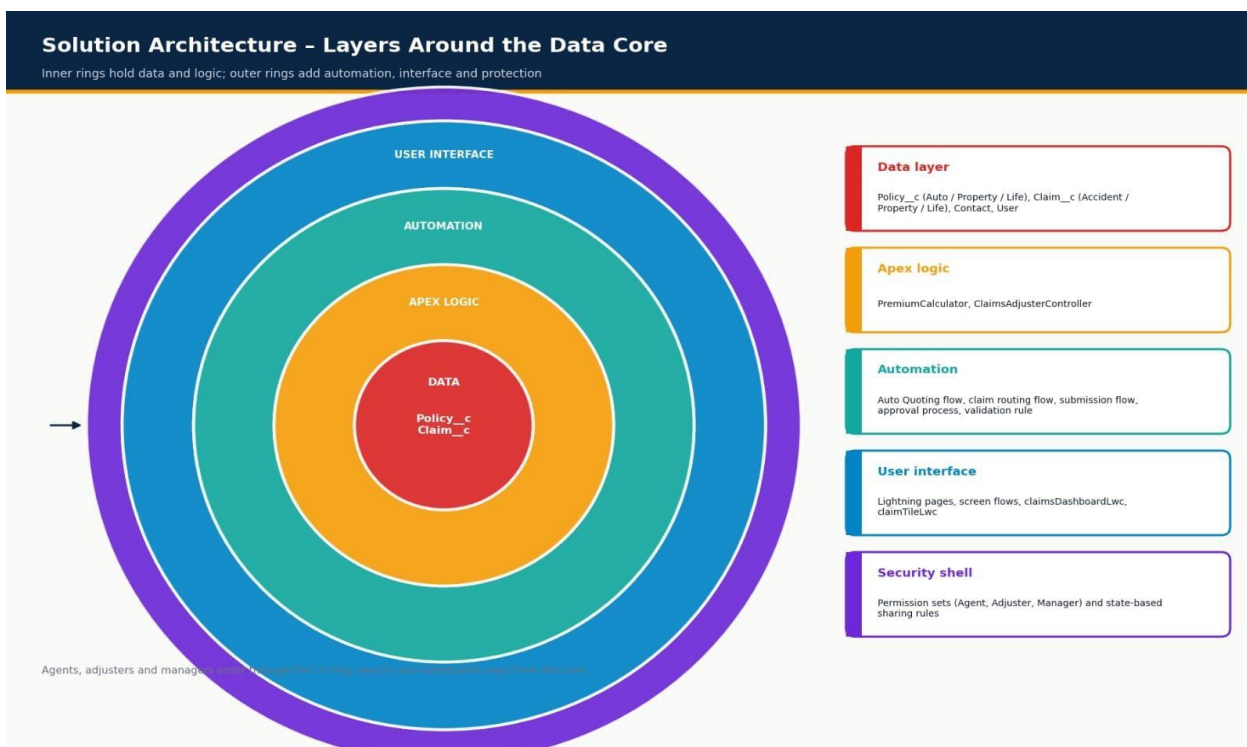


Figure 7: Layered solution architecture

Layer	Implementation
User layer	Insurance Agents, Claims Adjusters, Claims Managers
UI layer	Salesforce Lightning, Screen Flows, LWC
Automation layer	Record-Triggered Flows, Approval Processes, Validation Rules
Business logic	Apex Premium Calculator and Claims Controller
Data layer	Policy and Claim custom objects with Vehicle, Property and Life Record Types
Security layer	Permission Sets and state-based Sharing Rules
Reporting layer	Claims Dashboard, Reports and Salesforce Dashboards

5. Project Planning and Scheduling

5.1 Project Planning

- Agile methodology with sprint-based delivery.
- Work organised as Epics → User Stories → Story Points.
- Velocity is used to estimate future sprints.

Product Backlog and Sprint Schedule

Sprint	Epic	Story	User story / task	Pts	Priority	Member
1	Data Modeling	US-01	As an admin, I want to create Policy and Claim objects so insurance data is stored in an organised way.	5	High	Member 1
1	Data Modeling	US-02	As an admin, I want Vehicle, Property and Life Record Types so different products share one Policy structure.	3	High	Member 2

Sprint	Epic	Story	User story / task	Pts	Priority	Member
1	Data Modeling	US-03	As an admin, I want policy Field Sets and product-specific fields so varied insurance data is captured efficiently.	5	High	Member 3
1	Quoting & Automation	US-04	As an agent, I want a guided Auto Quoting Screen Flow so policy details are captured in a standard way.	8	High	Member 4
2	Validation	US-05	As an agent, I want VIN validation during vehicle quoting so wrong vehicle data is prevented.	3	High	Member 1
2	Apex	US-06	As the system, I want PremiumCalculator Apex to calculate premiums accurately.	8	High	Member 2
2	Automation	US-07	As an agent, I want premium calculation inside the quoting Flow so the premium is available during the quote.	5	High	Member 3
2	Claims Automation	US-08	As a claims manager, I want claims routed by policy type so they reach the right queue.	8	High	Member 4
3	Apex	US-09	As an adjuster, I want a ClaimsAdjusterController that returns my assigned claims so I can manage my workload.	8	High	Member 1
3	LWC Dashboard	US-10	As an adjuster, I want a claims dashboard and reusable claim tile to view and manage my claims.	8	High	Member 2

Sprint	Epic	Story	User story / task	Pts	Priority	Member
4	Approval	US-11	As a claims manager, I want high-value claims to go through an approval process and approver Flow for proper review.	8	High	Member 3
4	Security & Testing	US-12	As an admin, I want tests, sharing rules and Permission Sets so the solution is validated and access is correct.	8	High	Member 4

5.2 Project Tracker, Velocity and Burndown:

Sprint	Total points	Duration	Start	Planned end	Points done	Actual release
Sprint-1	20	6 days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 days	31 Oct 2022	05 Nov 2022		
Sprint-3	20	6 days	07 Nov 2022	12 Nov 2022		
Sprint-4	20	6days	14Nov 2022	19 Nov 2022		

6. Project Development Phase

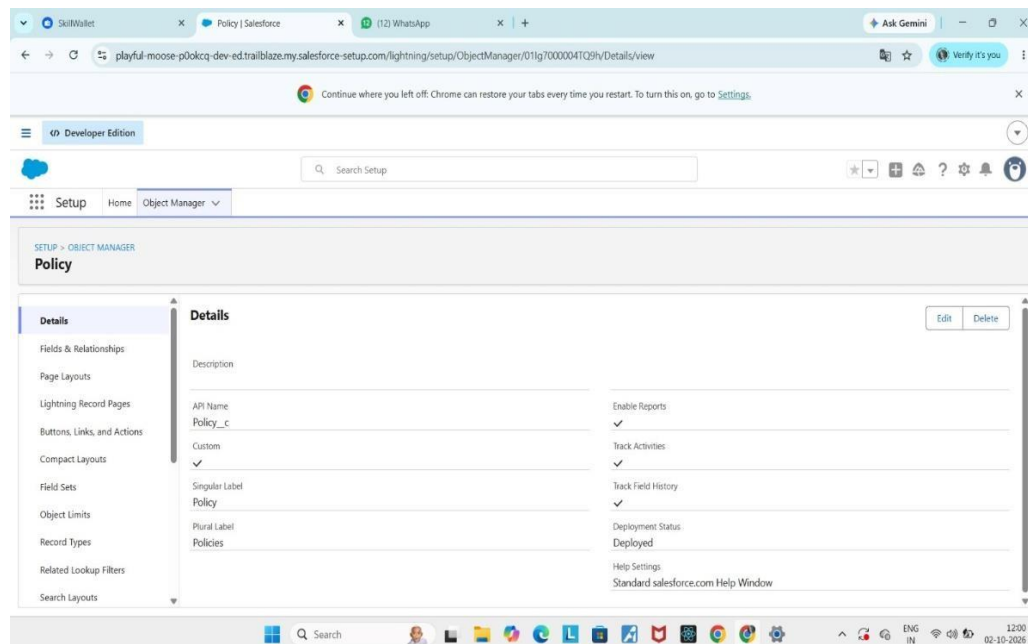
Topics covered:

- data modelling
- fields and relationships
- Lightning App Builder
- Record-Triggered Flows
- Apex and Apex triggers
- Batch Apex and Scheduled Apex.

Milestone 1: Core Data Model and Policy Configuration

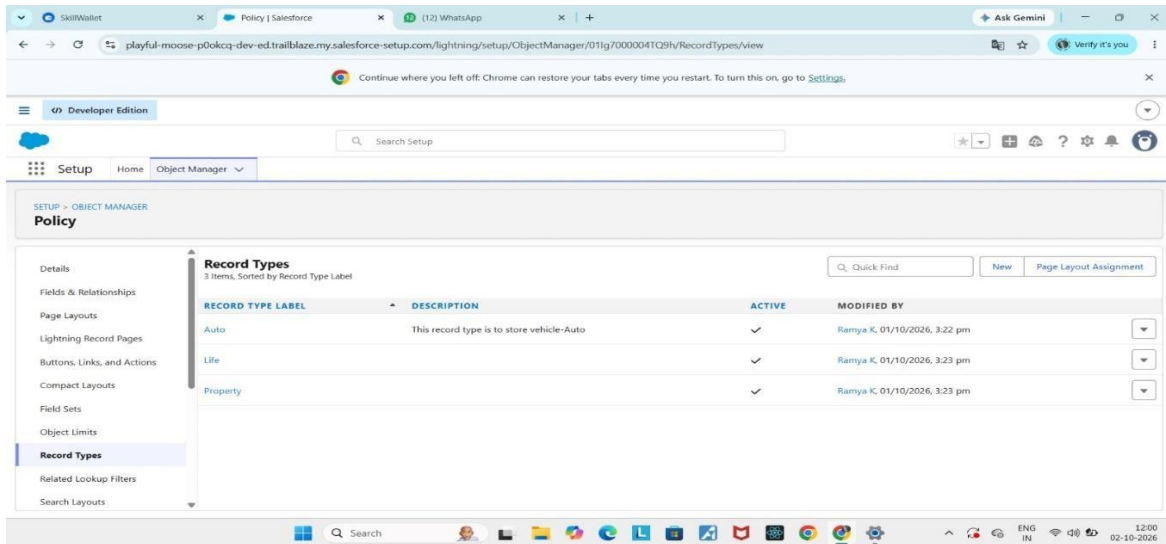
1. Create the Policy custom object

- Go to Setup → Object Manager → Create → Custom Object.
- Label: Policy, Plural Label: Policies. Object Name is auto-filled.
- Record Name: Policy Number, Data Type: Auto Number, Format: P-{0000}, Starting Number: 1
- Under optional features tick Allow Reports, Allow Activities and Track Field History.
- Set Deployment Status to Deployed and tick Allow Search, then Save.



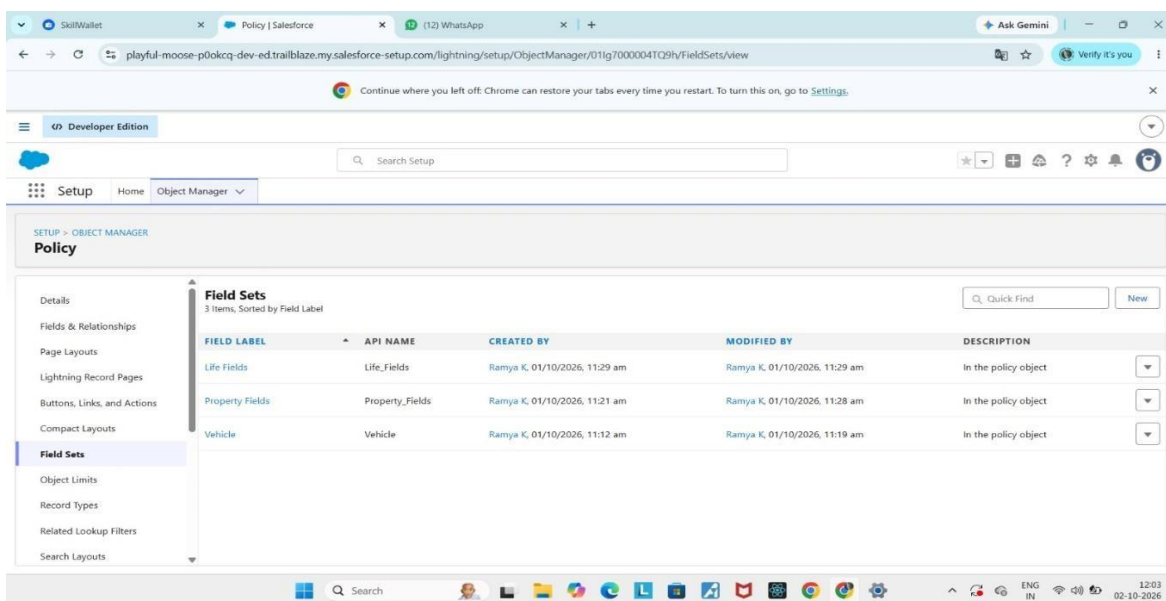
2. Create Record Types on the Policy object

- Object Manager → Policy__c → Record Types → New.
- Choose Master as existing record type, set the label to Auto, add a description and tick Active.
- Make it available to all profiles, click Next, accept the default page layout assignment and Save.
- Repeat the same steps for the Property and Life record types.



3. Create Field Sets for policy-specific data

- Object Manager → Policy__c → Field Sets → New.
- Label the first one Vehicle (used in the Policy object) and Save.
- Open it and drag VIN and Model Year into the set, then Save.
- Repeat for Property (Square Footage, Year Built) & Life (Beneficiary Name, Policy Term Months).



4. Create fields on the Policy object

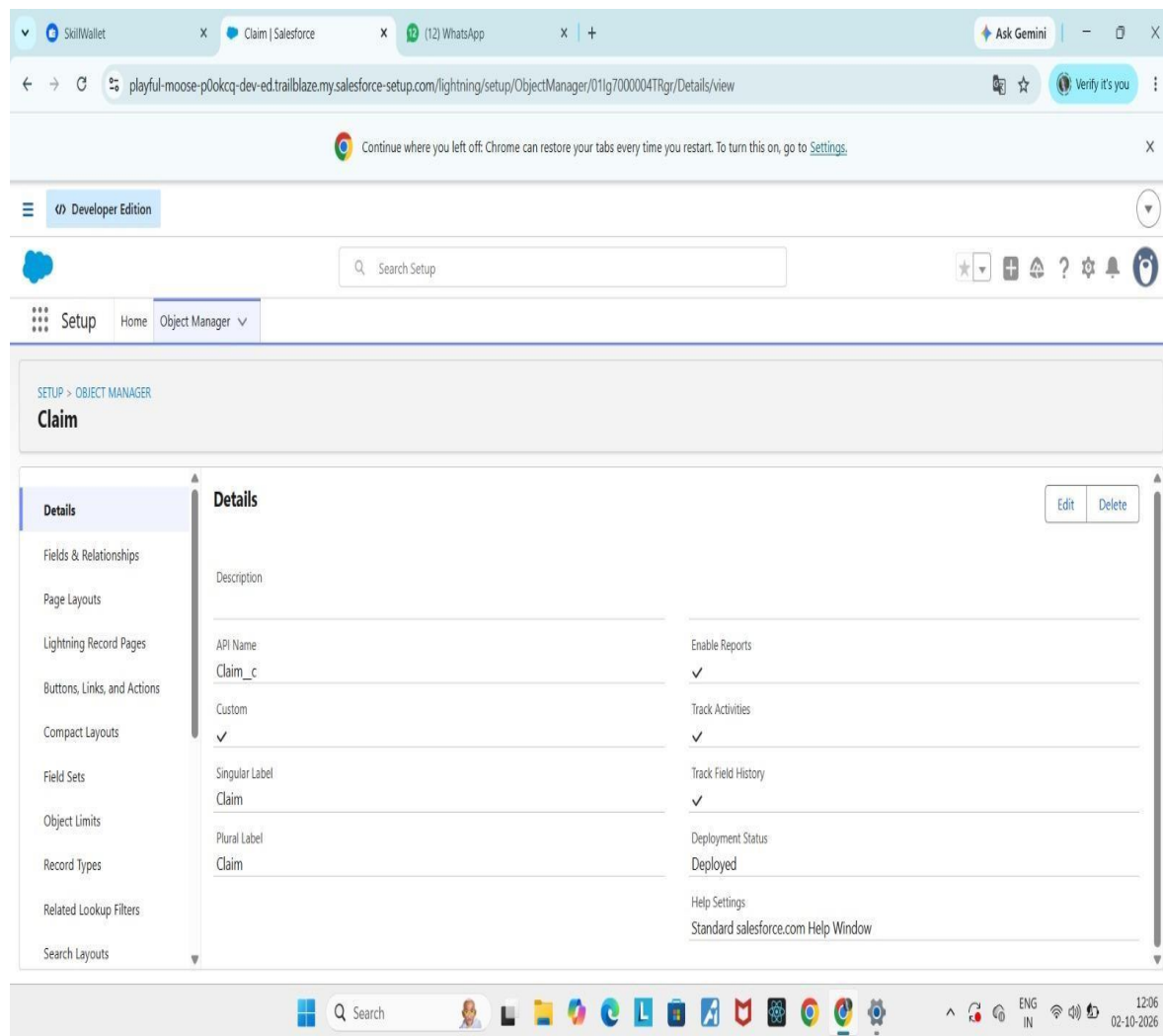
No.	Field name	Data type	Required
1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

The screenshot shows the Salesforce Lightning Setup page for the Policy object. The 'Fields & Relationships' section is active, displaying a table of fields and their properties. The table has columns for FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIELD, and INDEXED. The fields listed are:

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Beneficiary Name	Beneficiary Name, c	Text(50)		
Calculated Premium	Calculated Premium, c	Currency(16, 2)		
Created By	CreatedById	Lookup(Contact)		
Customer	Customer, c	Lookup(Contact)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Model Year	Model Year, c	Text(5)		
Owner	OwnerId	Lookup(Contact Group)		✓
Policy Number	Number	Auto Number		✓
Policy Start Date	Policy Start Date, c	Date		
Policy Date	Policy Date, c	Picklist		
Policy Term Months	Policy Term Months, c	Number(18, 0)		
Premium	Premium, c	Currency(16, 2)		
Record Type	RecordTypeId	Record Type		✓
Square Footage	Square Footage, c	Number(18, 0)		
VIN	VIN, c	Text(20)		
Year Built	Year Built, c	Text(5)		

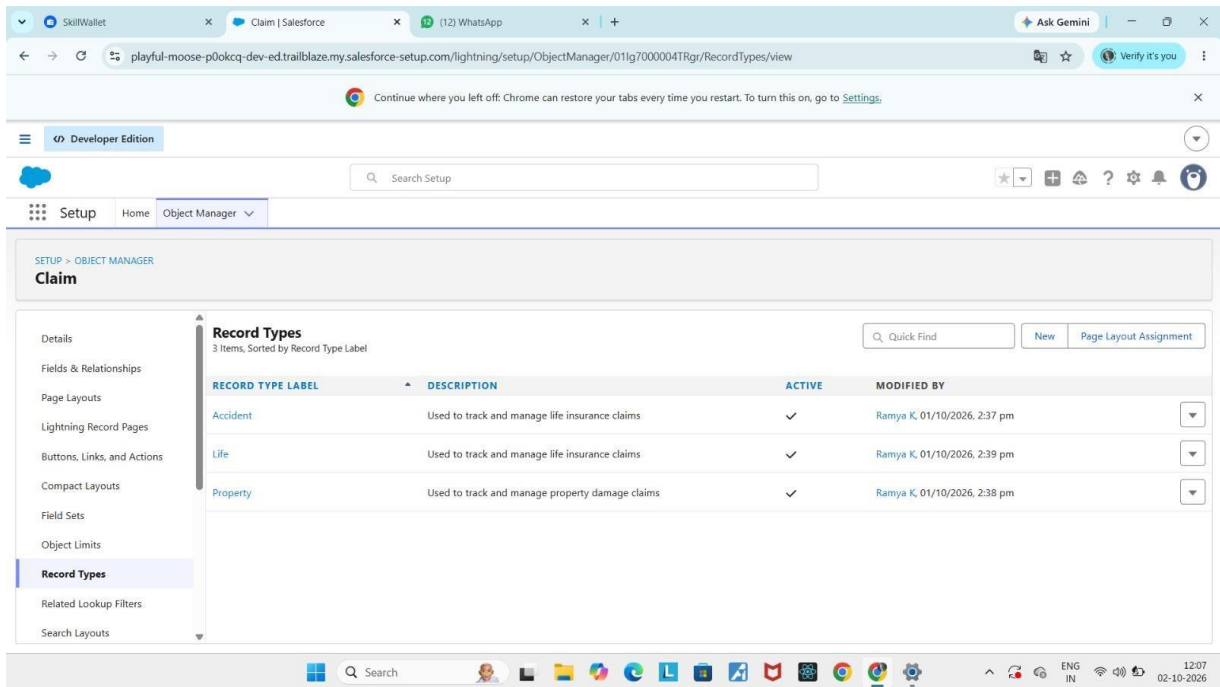
5. Create the Claim custom object

- Object Manager → Create → Custom Object.
- Label: Claim, Plural Label: Claim. Record Name: Claim Number, Data Type: Auto Number, Format: C-{0000}, Starting Number: 1.
- Tick Allow Reports, Allow Activities and Track Field History; set Deployed; tick Allow Search; & Save.



6. Create Record Types on the Claim object

- Claim__c → Record Types → New. Use Master as the base, label it Accident, add a description, tick Active, make it available to all profiles and Save.
- Repeat for the Property and Life record types.



7. Create fields on the Claim object

No.	Field name	Data type	Required
1	Adjuster	Lookup(User)	
2	Approval Status	Picklist (New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	
5	Description	Long Text Area	
6	Policy	Lookup(Policy)	

Setup > OBJECT MANAGER
Claim

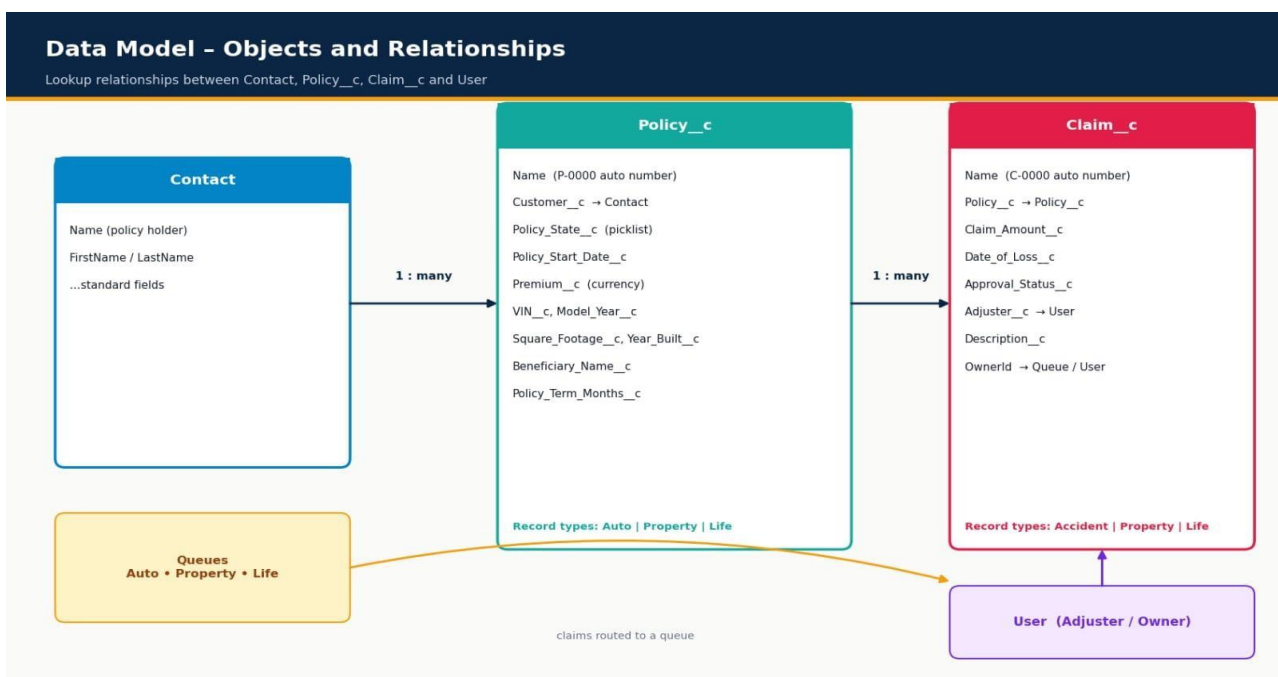
Details

Fields & Relationships
9 Items, Sorted by Field Label

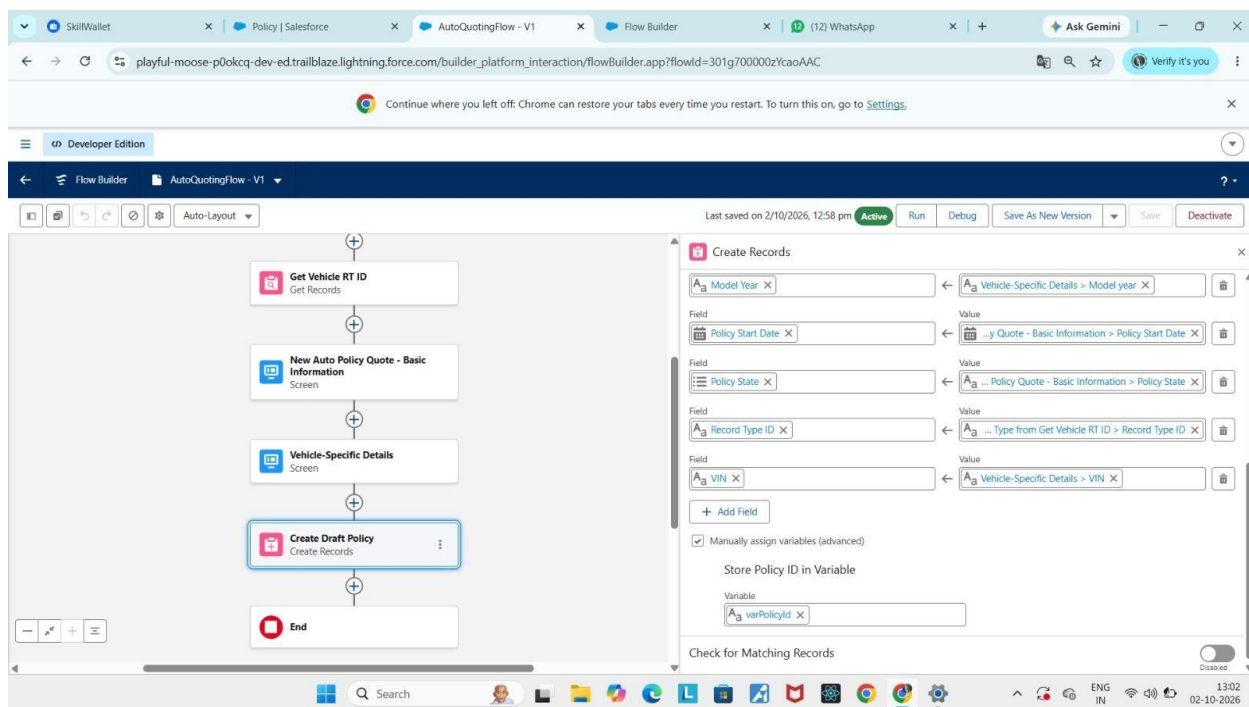
FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Approval Status	Approval_Status__c	Picklist		
Claim Amount	Claim_Amount__c	Currency(18, 0)		
Claim Number	Name	Auto Number		✓
Created By	CreatedById	Lookup(User)		
Date of Loss	Date_of_Loss__c	Date		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓

8. Build the AutoQuoting Screen Flow

Setup → Flows → New Flow → Screen Flow. Build the following elements in order:



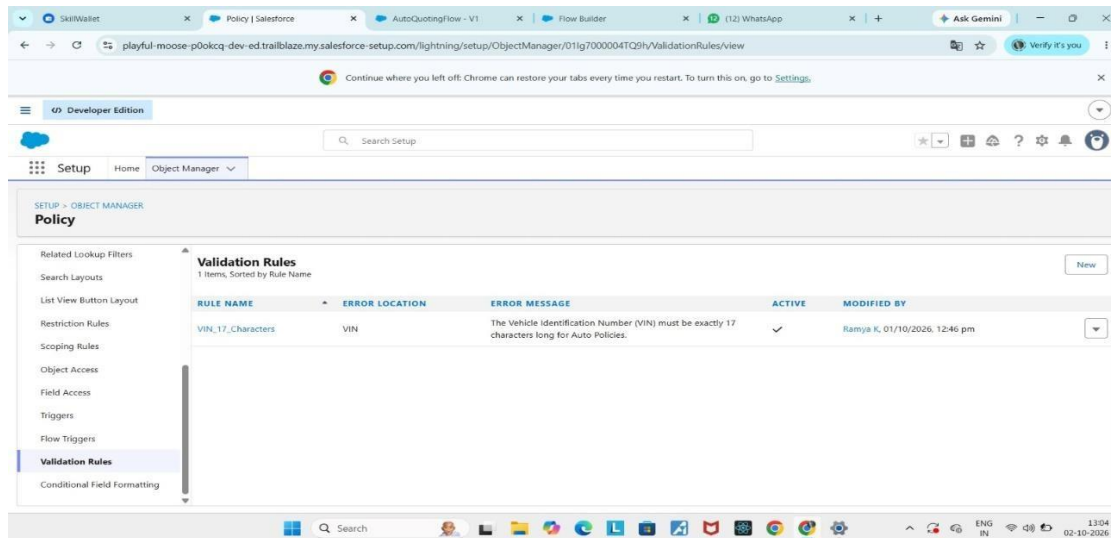
- **Get Records – "Get Vehicle RT ID"**: object Record Type; conditions: sObject Type Name equals Policy__c AND Record Type Name equals Auto.
- **Screen 1 – "New Auto Policy Quote - Basic Information"**: add a Lookup (API name Customer Lookup, field Customer__c, label Policy Holder, object Policy__c), a Date field (Policy Start Date) and a Picklist (Policy State) backed by a new Picklist Choice Set on Policy → Policy State.
- **Screen 2 – "Vehicle-Specific Details"**: add two text inputs, VIN and Model Year.
- **Create Records – "Create Draft Policy"**: set Customer from the Policy Holder record ID, Model Year, Policy Start Date, Policy State, VIN, and RecordTypeId from Get Vehicle RT ID. Store the new record ID in a Text variable named varPolicyId. Save the flow as AutoQuotingFlow and activate it.



9. Create the VIN validation rule

- Object Manager → Policy → Validation Rules → New.
- Rule name: VIN_Must_Be_17_Characters, tick Active.
- Formula: `AND( RecordType.DeveloperName = "Auto", LEN( VIN__c ) <> 17 )`

- Error message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Show the error on the VIN field and Save.



Milestone 2: Complex Policy Issuance and Claim Routing Automation

1. Create the PremiumCalculator Apex class

Open Developer Console → File → New → Apex Class, name it PremiumCalculator and paste the code below. Press Ctrl+S to save. The class is called from AutoQuotingFlow.

```
public class PremiumCalculator {

    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(label='Calculate Policy Premium'
        description='Calculates premium using VIN, Model Year and State.')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
```

```
List<Decimal> premiumResults = new List<Decimal>();  
Set<Id> policyIds = new Set<Id>();
```

```
for (PremiumInputWrapper input : inputs) {  
    if (input.policyId != null) {  
        policyIds.add(input.policyId);  
    }  
}
```

```
List<Policy__c> policiesToRate = [  
    SELECT Id, VIN__c, Model_Year__c, Policy_State__c  
    FROM Policy__c  
    WHERE Id IN :policyIds  
];
```

```
for (Policy__c policy : policiesToRate) {  
    Decimal basePremium = 1000.00;  
  
    if (policy.Policy_State__c == 'CA') {  
        basePremium *= 1.15;  
    } else if (policy.Policy_State__c == 'TX') {  
        basePremium *= 1.05;  
    }  
}
```

```
if (policy.Model_Year__c != null &&  
    Integer.valueOf(policy.Model_Year__c) < 2018) {  
    basePremium += 100;  
}
```

```
premiumResults.add(basePremium);  
}
```

```

    return premiumResults;
}
}

```

```

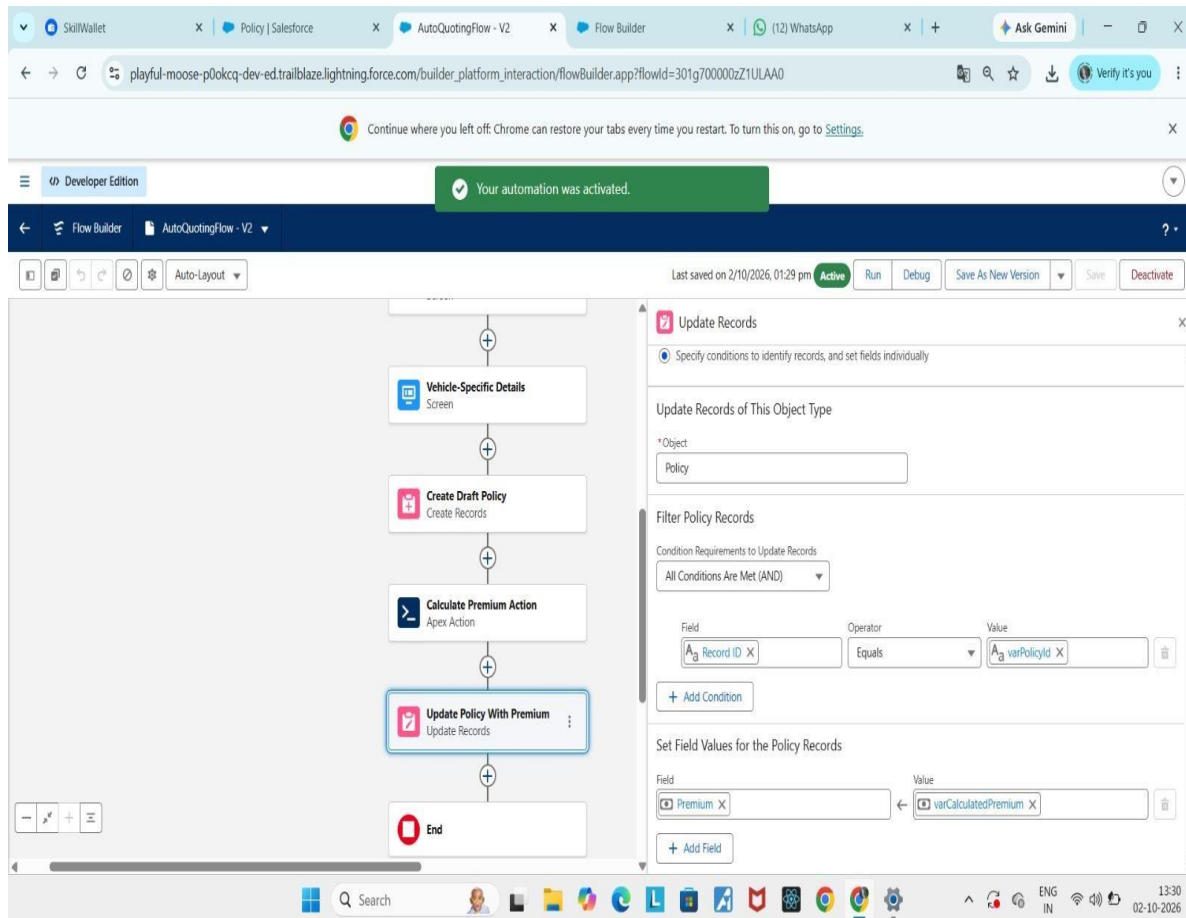
1 public class PremiumCalculator {
2     public class PremiumInputWrapper {
3         @InvocableVariable(required=true)
4         public Id policyId;
5     }
6
7     @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on policy fields (VIN, Model Year, State).')
8     public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
9
10        List<Decimal> premiumResults = new List<Decimal>();
11        Set<Id> policyIds = new Set<Id>();
12
13        for (PremiumInputWrapper input : inputs) {
14            if (input.policyId != null) {
15                policyIds.add(input.policyId);
16            }
17        }
18
19        List<Policy__c> policiesToRate = [
20            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
21            FROM Policy__c
22            WHERE Id IN :policyIds
23        ];
24
25        for (Policy__c policy : policiesToRate) {
26            Decimal basePremium = 1000.00;
27
28            if (policy.Policy_State__c == 'CA') {
29                basePremium *= 1.15;
30            } else if (policy.Policy_State__c == 'TX') {
31                basePremium *= 1.05;
32            }
33        }
34    }
35 }

```

2. Call the Apex action from AutoQuotingFlow

- Open AutoQuotingFlow in Flow Builder.
- After Create Records add an Action element labelled "Calculate Premium Action"; pass varPolicyId as the policy Id.
- For the output, create a Currency variable varCalculatedPremium with 2 decimal places.

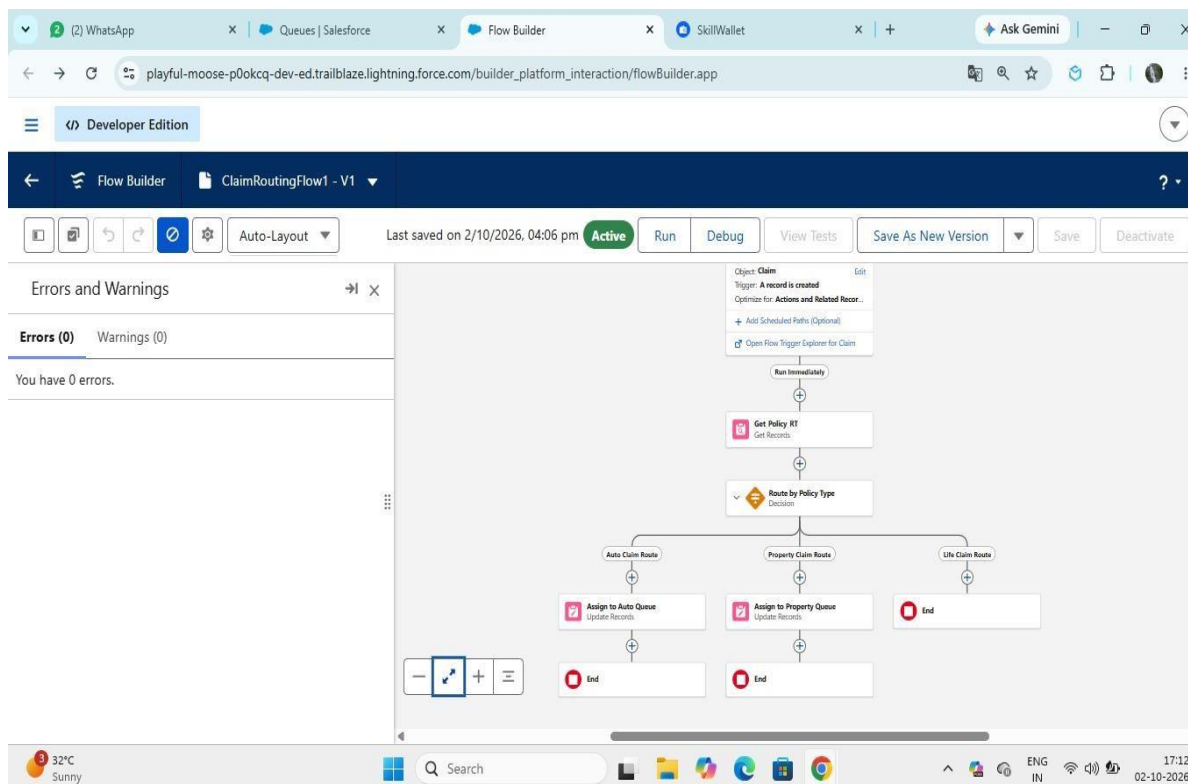
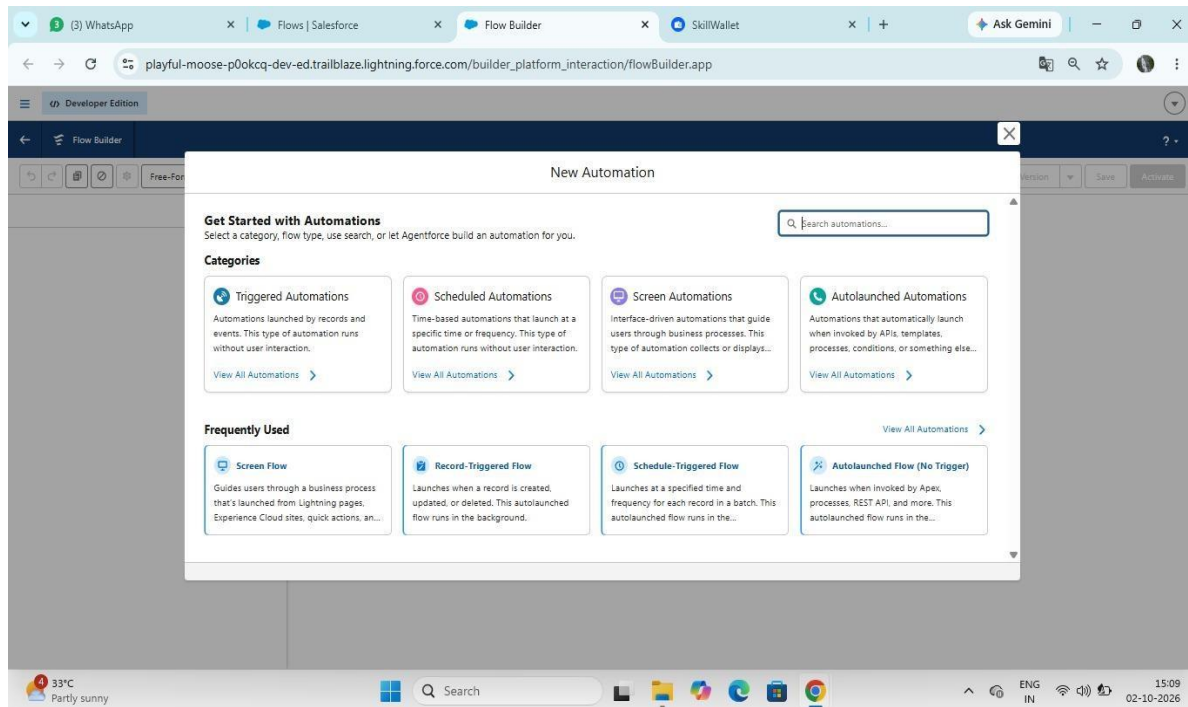
- Add an Update Records element, "Update Policy With Premium": find the policy where Record Id equals varPolicyId and set Premium to varCalculatedPremium.
- Save as a new version and activate.



3. Create the claim routing Record-Triggered Flow (After Save)

- New Flow → Record-Triggered Flow; object Claim; trigger when a record is created; optimise for Actions and Related Records.
- Get Records "Get_Policy_RT": object Policy, where Record Id equals the triggering Claim's Policy; store the first record, and put the record type into a Text variable varPolicyRTName.
- Decision "Route by Policy Type": outcome "Auto Claim Route" when varPolicyRTName equals Auto; outcome "Property Claim Route" when it equals Property; default outcome "Life Claim Route".

- On each path add an Update Records element ("Assign to Auto Queue", "Assign to Property Queue", "Assign to Life Queue") that finds the triggering Claim by Record Id and sets Owner ID to the 18-digit ID of the matching queue.



Milestone 3: Claims Adjuster LWC Dashboard

1. Create the ClaimsAdjusterController Apex class

This class queries the claims owned by the logged-in user, joins the related Policy and Customer data, and returns a wrapper list with the key metrics.

```
public class ClaimsAdjusterController {

    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id;
            this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c;
            this.status = c.Approval_Status__c;

            if (p != null && p.RecordType != null) {
                this.policyType = p.RecordType.Name;
            } else {
                this.policyType = 'Unknown';
            }

            this.policyHolderName = (customerRec != null) ? customerRec.Name : '';

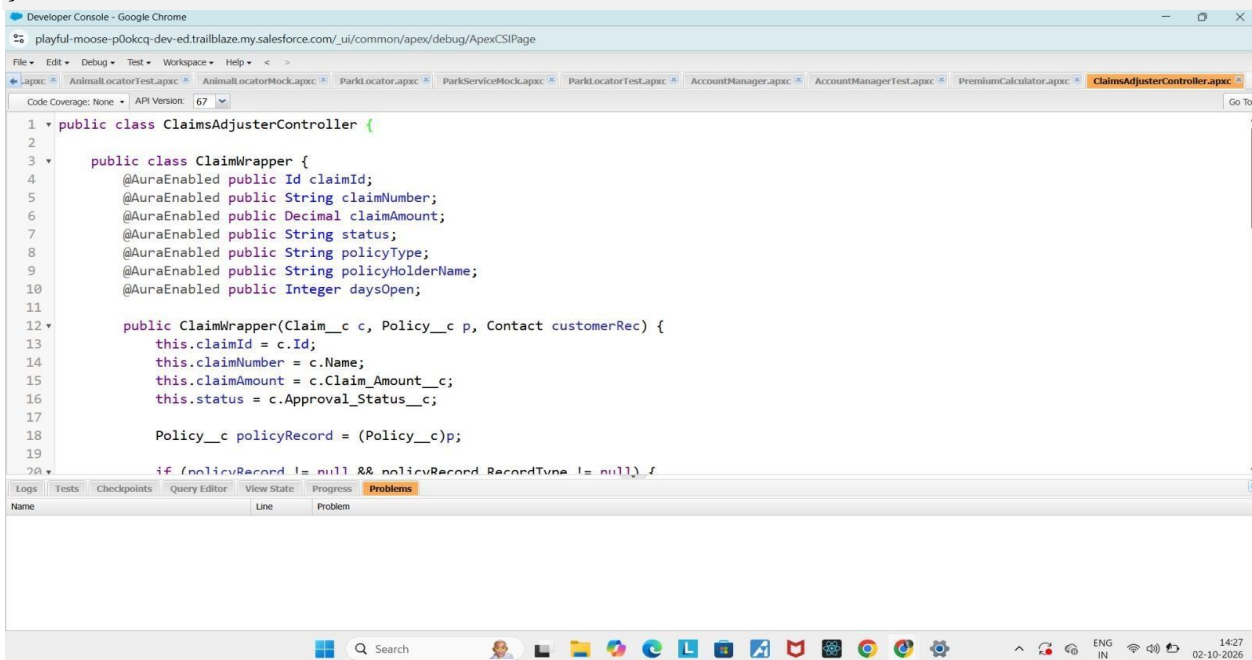
            // days elapsed since the date of loss
            this.daysOpen = (c.Date_of_Loss__c != null)
                ? c.Date_of_Loss__c.date().daysBetween(Date.today())
                : 0;
        }
    }

    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {
```

```
List<ClaimWrapper> results = new List<ClaimWrapper>();
```

```
List<Claim__c> claims = [
    SELECT Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
        Policy__r.Name,
        Policy__r.RecordType.Name,
        Policy__r.RecordType.DeveloperName,
        Policy__r.Customer__r.FirstName,
        Policy__r.Customer__r.LastName
    FROM Claim__c
    WHERE OwnerId = :UserInfo.getUserId()
    ORDER BY Date_of_Loss__c DESC
];
```

```
for (Claim__c claim : claims) {
    Policy__c policy = claim.Policy__r;
    Contact customer = (policy != null && policy.Customer__r != null)
        ? (Contact) policy.Customer__r
        : new Contact();
    results.add(new ClaimWrapper(claim, policy, customer));
}
return results;
}
```



2. Create the claimsDashboardLwc component

Using the Lightning Studio browser extension, choose Create New → Lightning Web Component, name it claimsDashboardLwc, tick isExposed, select the App, Home and Record page targets and Deploy. Then paste the code below into each file.

claimsDashboardLwc.html

```
<template>
  <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">
    <div class="slds-m-around_medium">
      <lightning-combobox
        name="policyFilter"
        label="Filter by Policy Type"
        value="All"
        options={policyTypeOptions}
        onchange={handleFilterChange}
        class="slds-m-bottom_medium">
      </lightning-combobox>

      <template if:true={visibleClaims}>
        <div class="slds-grid slds-wrap slds-grid_pull-padded">
          <template for:each={visibleClaims} for:item="claim">
            <div key={claim.claimId}
              class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-
p-horizontal_small">
              <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
            </div>
          </template>
        </div>
        <p class="slds-m-top_medium slds-text-heading_small">
          Total Claims Assigned: {visibleClaims.length}
        </p>
      </template>

      <template if:false={visibleClaims}>
        <p>No claims currently assigned to you.</p>
      </template>
    </div>
  </lightning-card>
</template>
```

claimsDashboardLwc.js

```
import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
  @track allClaims = []; // everything returned by Apex
  @track visibleClaims = []; // what is shown after filtering
  error;

  @wire(getAssignedClaims)
  wiredClaims({ error, data }) {
    if (data) {
      this.allClaims = data;
      this.visibleClaims = data;
      this.error = undefined;
    } else if (error) {
      this.error = error;
      this.allClaims = undefined;
      this.visibleClaims = undefined;
      console.error('Error retrieving claims:', JSON.stringify(error));
    }
  }

  // client-side filtering
  handleFilterChange(event) {
    const selected = event.target.value;
    if (selected === 'All') {
      this.visibleClaims = this.allClaims;
    } else {
      this.visibleClaims = this.allClaims.filter(
        claim => claim.policyType === selected
      );
    }
  }

  get policyTypeOptions() {
    return [
      { label: 'All Policy Types', value: 'All' },
      { label: 'Auto', value: 'Auto' },
      { label: 'Property', value: 'Property' },
    ];
  }
}
```

```

    { label: 'Life', value: 'Life' }
  ];
}
}

```

claimsDashboardLwc.js-meta.xml

```

<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>

```

The screenshot displays the Lightning Studio IDE environment. The top bar shows the 'LIGHTNING STUDIO' title and various toolbars. The left sidebar contains a 'Search Component...' field and a list of 'LIGHTNING WEB COMPONENTS' (21/21), including 'barcodeScanner', 'brokerCard', 'claimsdashboaedlwc', 'claimsDashboardLwc', 'claimTileLwc', 'daysOnMarket', 'errorPanel', 'helloWorld', 'ldsUtils', 'listContactsFromDevice', 'navigateToRecord', 'paginator', 'propertyCarousel', 'propertyFilter', 'propertyListMap', 'propertyLocation', 'propertyMap', 'propertySummary', 'propertyTile', and 'propertyTileList'. The main editor area shows the 'claimsdashboaedlwc.js-meta.xml' file, which contains the following XML structure:

```

<template>
  <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">
    <div class="slds-m-around_medium">
      <lightning-combobox
        name="policyFilter"
        label="Filter by Policy Type"
        value="All"
        options={policyTypeOptions}
        onchange={handleFilterChange}
        class="slds-m-bottom_medium"
      ></lightning-combobox>
    </div>
    <template if:true={visibleClaims}>
      <div class="slds-grid slds-wrap slds-grid_pull-padded">
        <template for:each={visibleClaims} for:item="claim">
          <div key={claim.claimId} class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3">
            <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
          </div>
        </template>
      </div>
      <p class="slds-m-top_medium slds-text-heading_small">Total Claims Assigned: {visibleClaims.length}</p>
    </template>
  </lightning-card>
</template>

```

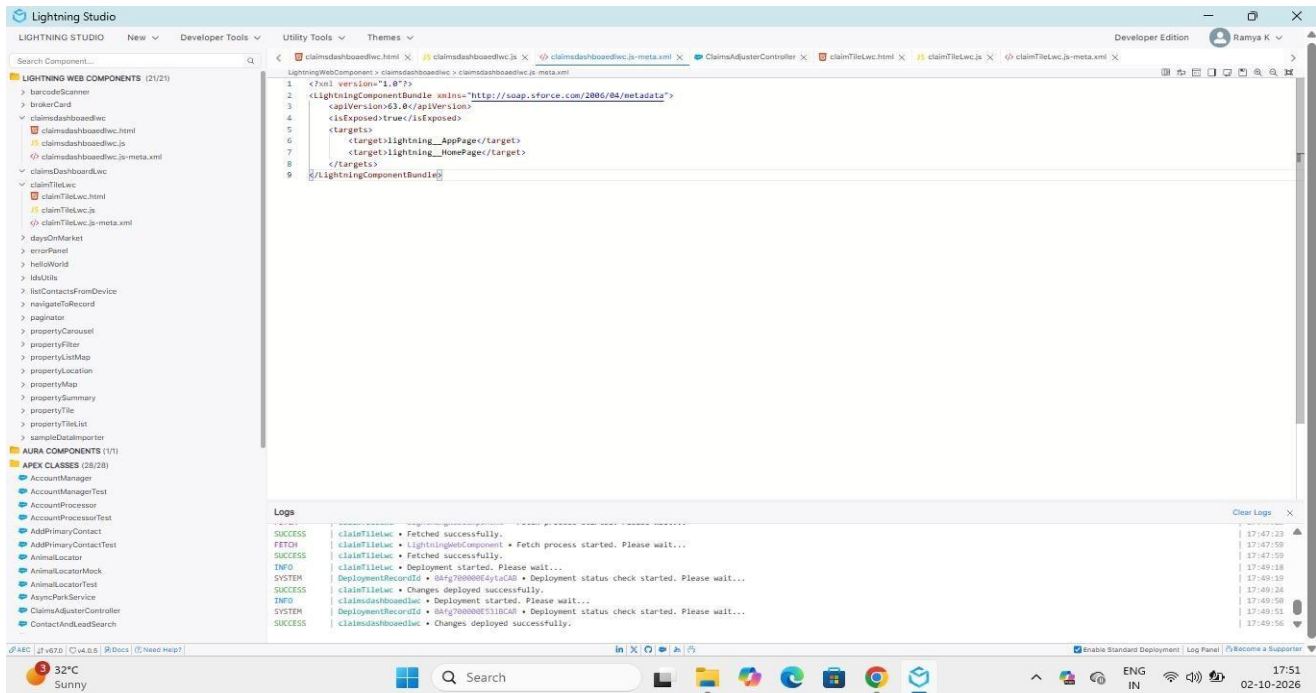
The bottom panel shows the 'Logs' window, which displays deployment status messages:

```

INFO    claimTileLwc • Deployment started. Please wait... 17:49:18
SYSTEM  DeploymentRecordId • 0Afg700000E4ytaCAB • Deployment status check started. Please wait... 17:49:19
SUCCESS claimTileLwc • Changes deployed successfully. 17:49:24
INFO    claimsdashboaedlwc • Deployment started. Please wait... 17:49:50
SYSTEM  DeploymentRecordId • 0Afg700000E53lBCAR • Deployment status check started. Please wait... 17:49:51
SUCCESS claimsdashboaedlwc • Changes deployed successfully. 17:49:56

```

The bottom status bar shows the current version (v67.0), the user (Ramy K), and the date (02-10-2026).



3. Create the reusable claimTileLwc component

Create it the same way, name it claimTileLwc, tick isExposed and Deploy.

claimTileLwc.html

```
<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media__figure">
          <lightning-icon icon-name={policyIcon}
            alternative-text={claimData.policyType} size="small">
          </lightning-icon>
        </div>
        <div class="slds-media__body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}
          </h2>
          <p class="slds-text-body_small">
            Policy Type: <strong>{claimData.policyType}</strong>
          </p>
        </div>
      </header>
    </div>
  </lightning-layout-item>
</template>
```

```

<div class="slds-border_top slds-p-top_small">
  <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p>
  <lightning-formatted-number
    value={claimData.claimAmount}
    format-style="currency"
    currency-code="USD"
    label="Claim Amount">
  </lightning-formatted-number>
  <p>Status: {claimData.status}</p>
  <p class="slds-text-body_regular">
    Days Open:
    <strong class="slds-text-color_error">{claimData.daysOpen}</strong>
  </p>
</div>
</div>
</lightning-layout-item>
</template>

```

claimTileLwc.js

```

import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
  @api claimData; // supplied by the parent dashboard

  get policyIcon() {
    if (this.claimData && this.claimData.policyType) {
      const type = this.claimData.policyType.toLowerCase();
      if (type === 'auto') return 'utility:truck';
      if (type === 'property') return 'utility:home';
      if (type === 'life') return 'utility:connected_apps';
    }
    return 'utility:help';
  }
}

```

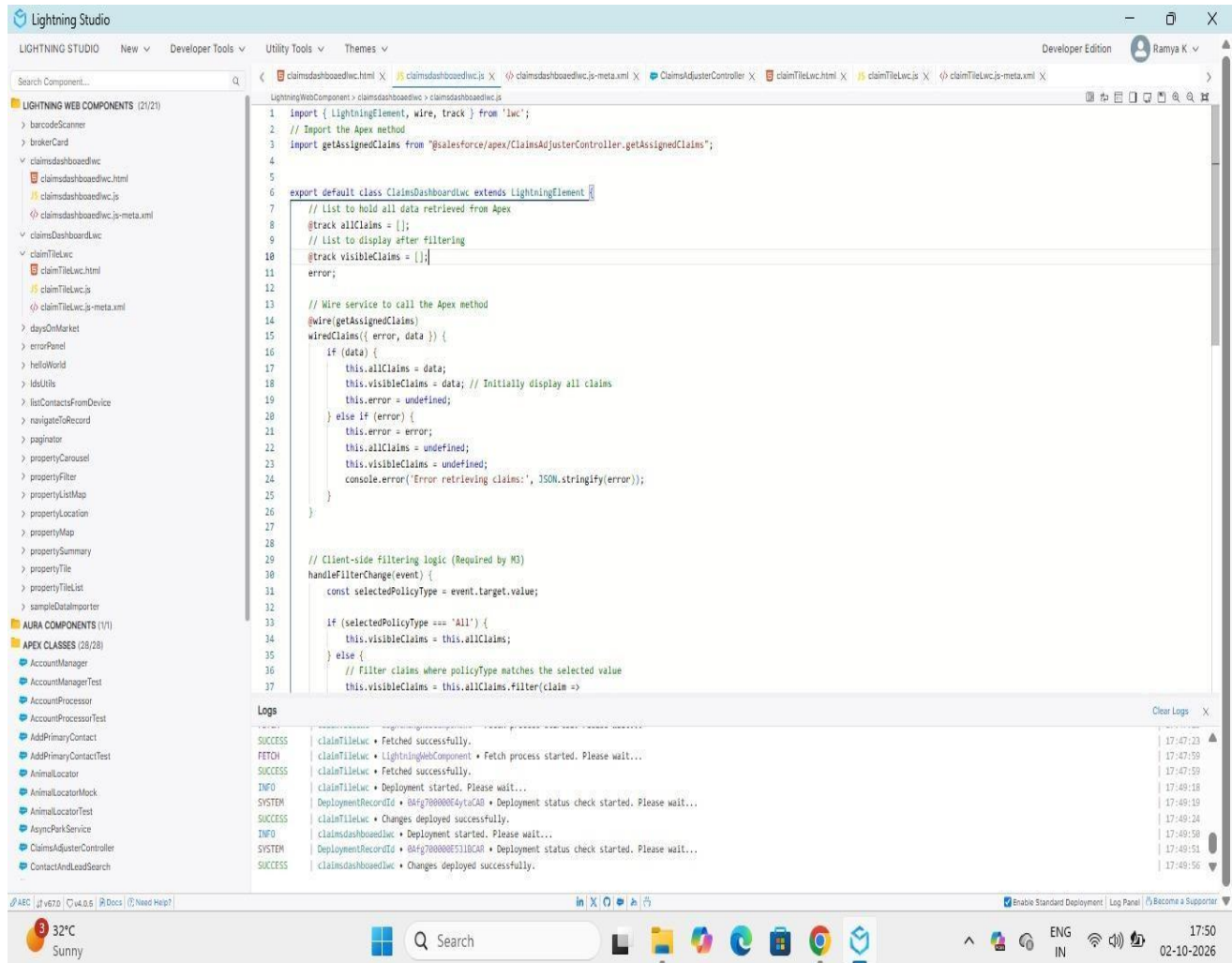
claimTileLwc.js-meta.xml

```

<?xml version="1.0"?>

```

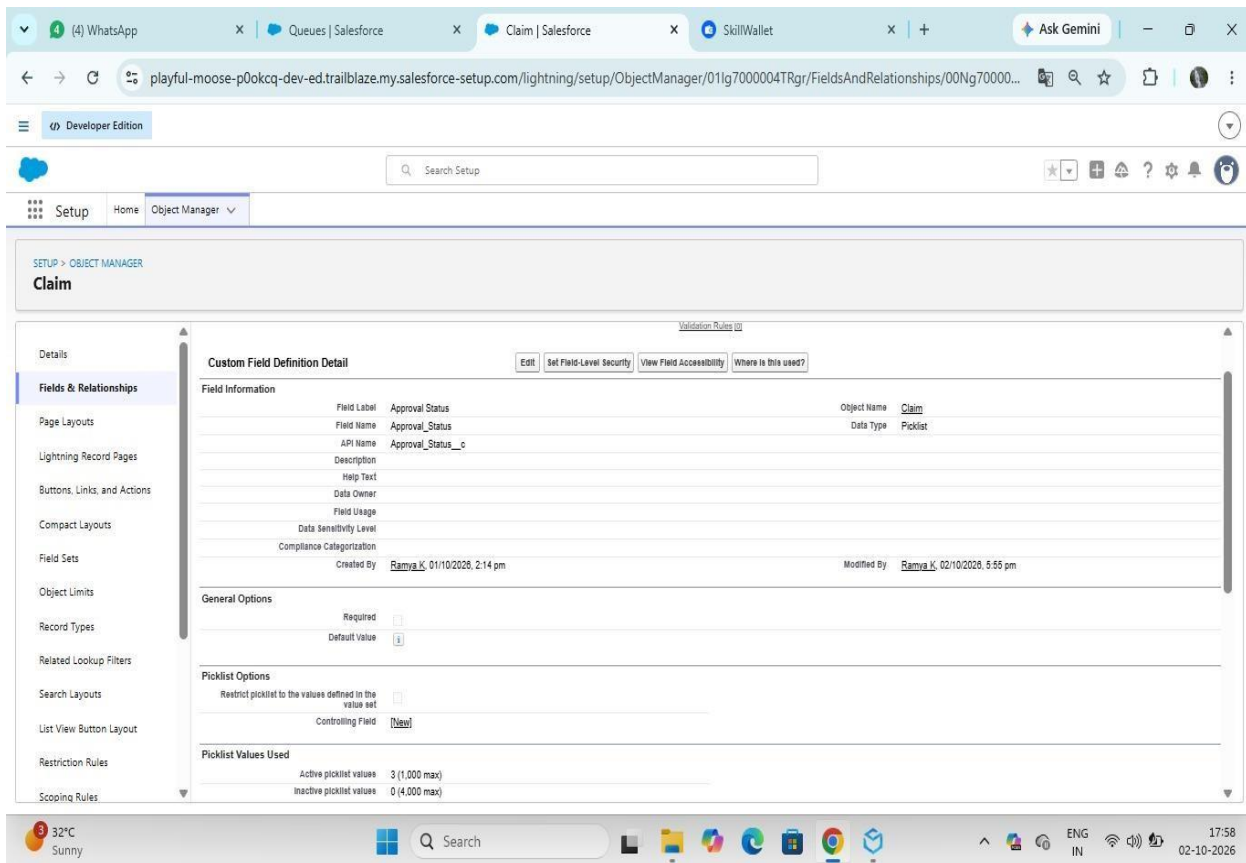
```
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
</LightningComponentBundle>
```



Milestone 4: Advanced Claim Processing, Security and Code Setup

1. Add the Approval Status field to Claim

- Object Manager → Claim → Fields & Relationships → New → Picklist.
- Label: Approval Status. Enter the values New, Submitted for Approval, Approved and Rejected, one per line, then Next, Next, Save.



2. Configure the High Value Claim Approval Process

- Setup → Approval Processes → manage for Claim → Create New → Standard Setup Wizard.
- Name: High Value Claim Approval. Entry criteria: Claim Amount greater than 50000.
- Record editability: Administrators ONLY can edit records during approval.
- Approval page layout: add Claim Number, Owner, Claim Amount, Adjuster and Policy; allow approvers to access the page only from within Salesforce.
- Initial submitters: add the Claim Owner. Save.

2.1 Approval steps

- **Step 1 – "Step1 Senior Adjuster"**: all records enter the step; automatically assign to the Senior Adjuster user; with multiple approvers, decide on the first response.

- **Step 2 – "Step2 Manager Review"**: all records enter the step; automatically assign to the Manager user; decide on the first response; on rejection perform all rejection actions for the step and all final rejection actions.

Add three Field Update actions on the Claim object, each updating Approval Status (picklist) to a specific value:

- **Initial Submission Action** – "Update Status Submitted" sets Submitted for Approval.
- **Final Approval Action** – "Update Status Approved" sets Approved.
- **Final Rejection Action** – "Update Status Rejected" sets Rejected.

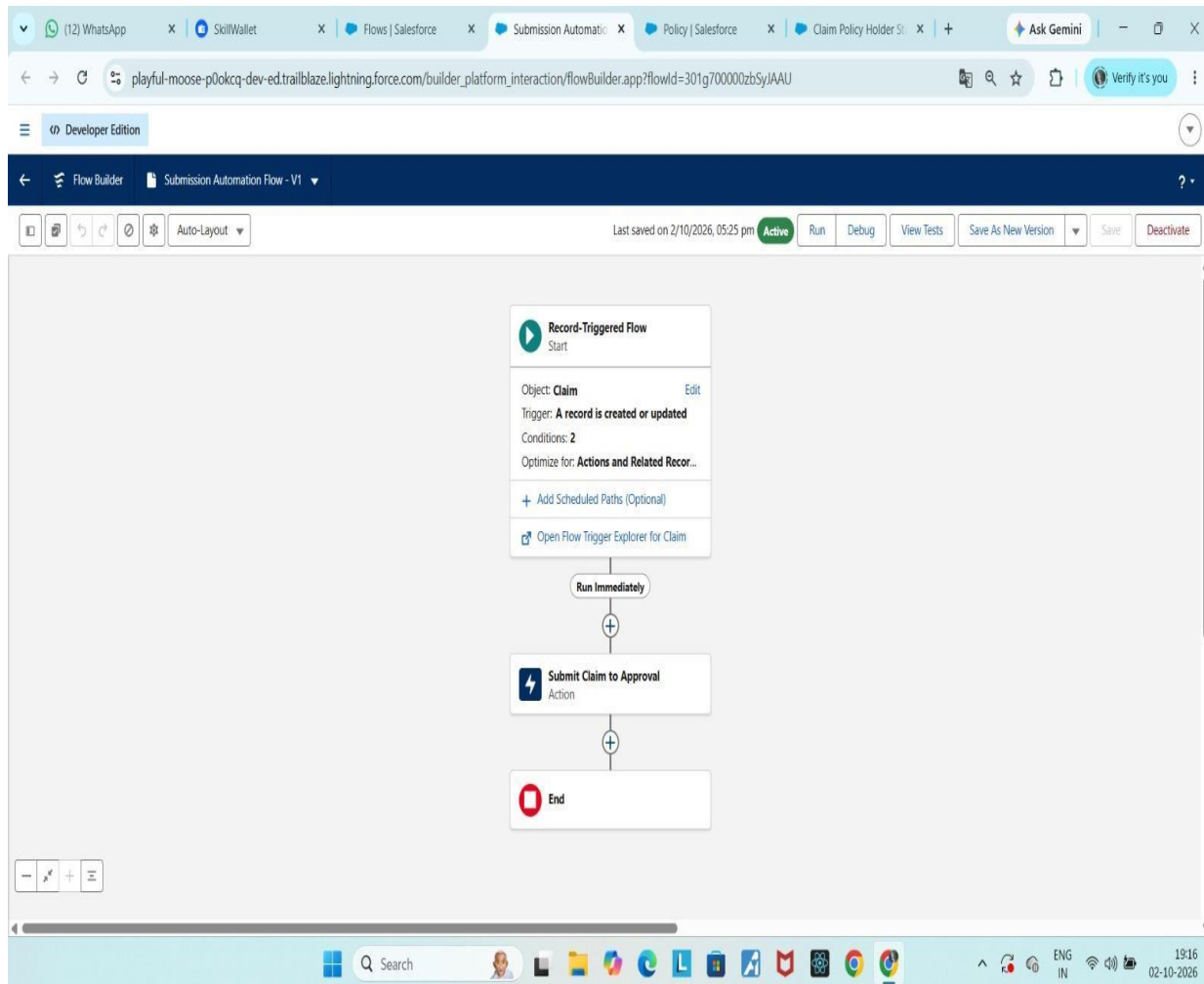
Activate the approval process.

The screenshot shows the Salesforce Setup interface for an Approval Process. The left sidebar contains navigation links like 'Setup', 'Home', 'Object Manager', and a search bar. The main content area is titled 'Approval Processes' and shows the configuration for the 'High Value Claim Approval' process. The process is active and has a 'Next Automated Approver Determined By' field set to 'Administrator ONLY'. Below this, there are sections for 'Initial Submission Actions', 'Approval Steps', 'Final Approval Actions', 'Final Rejection Actions', and 'Recall Actions'. Each section contains a table of actions with columns for 'Action', 'Type', and 'Description'. The 'Approval Steps' section shows two steps: 'Step1 Senior Adjuster' and 'Step2 Manager Review'. The 'Final Approval Actions' section shows an action to 'Update Status Approved'. The 'Final Rejection Actions' section shows an action to 'Update Status Rejected'. The 'Recall Actions' section shows an action to 'Unlock the record for editing'.

3. Create the submission Record-Triggered Flow (After Save)

- New Flow → Record-Triggered Flow on Claim; run when a record is created or updated.
- Conditions (all true): Claim Amount greater than 50000 AND Approval Status equals New. Run every time a record is updated and meets the conditions; optimise for Actions and Related Records.

- Add an Action "Submit Claim to Approval": Record Id = triggering Claim → Record ID; approval process = High Value Claim Approval.
- Save as Submission Automation Flow and activate.

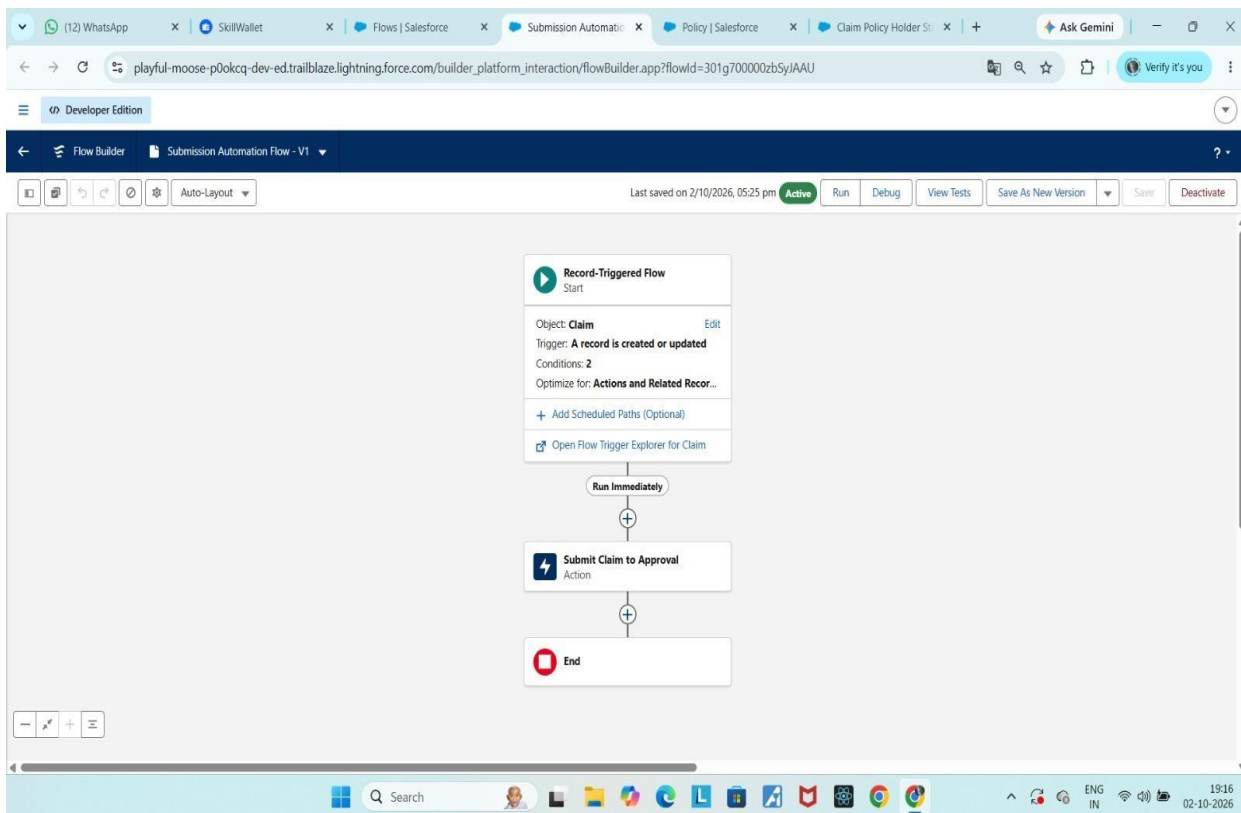


4. Create the approver Screen Flow

- New Screen Flow. Create a Text variable recordId and tick Available for Input.
- Get Records "Get Claim Details": object Claim where RecordId equals recordId; first record only; store all fields.
- Screen "Claim Review Screen": Display Text showing Claim ID, Amount and Policy Type from the fetched record; a Radio Buttons component "Approval Decision" with two Text choices, Approve and Reject; and a Long Text Area "Approver Comments".
- Decision "Check Decision": outcome "Decision Approved" when Approval Decision equals

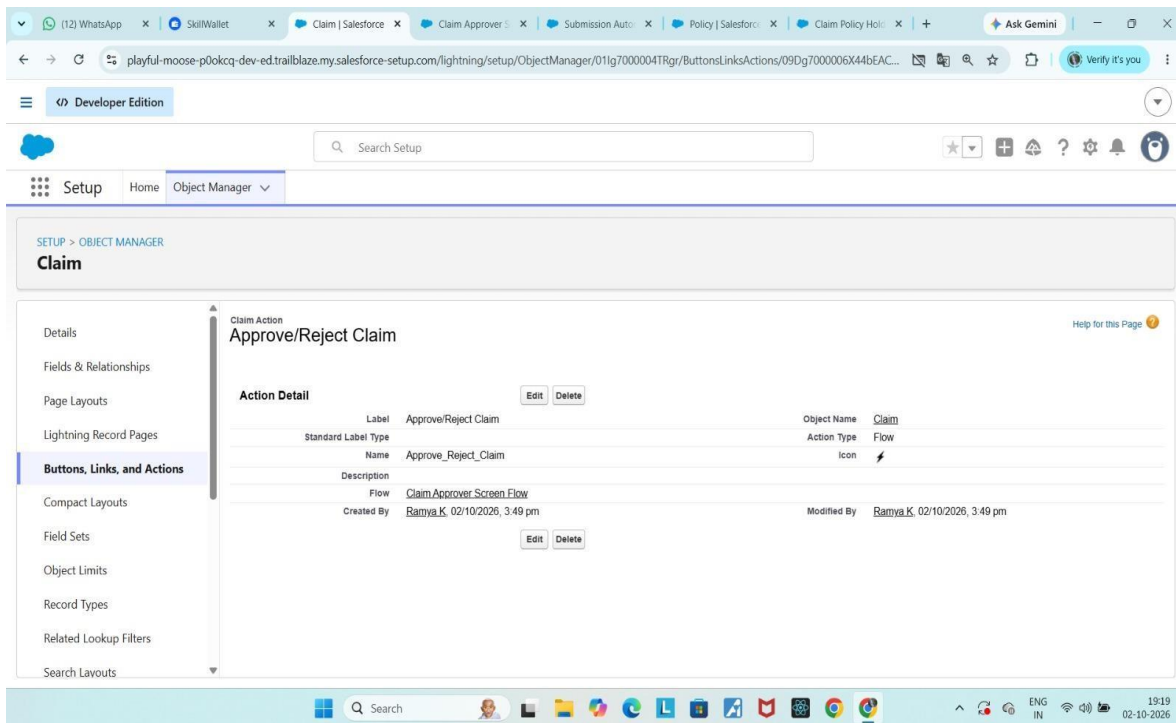
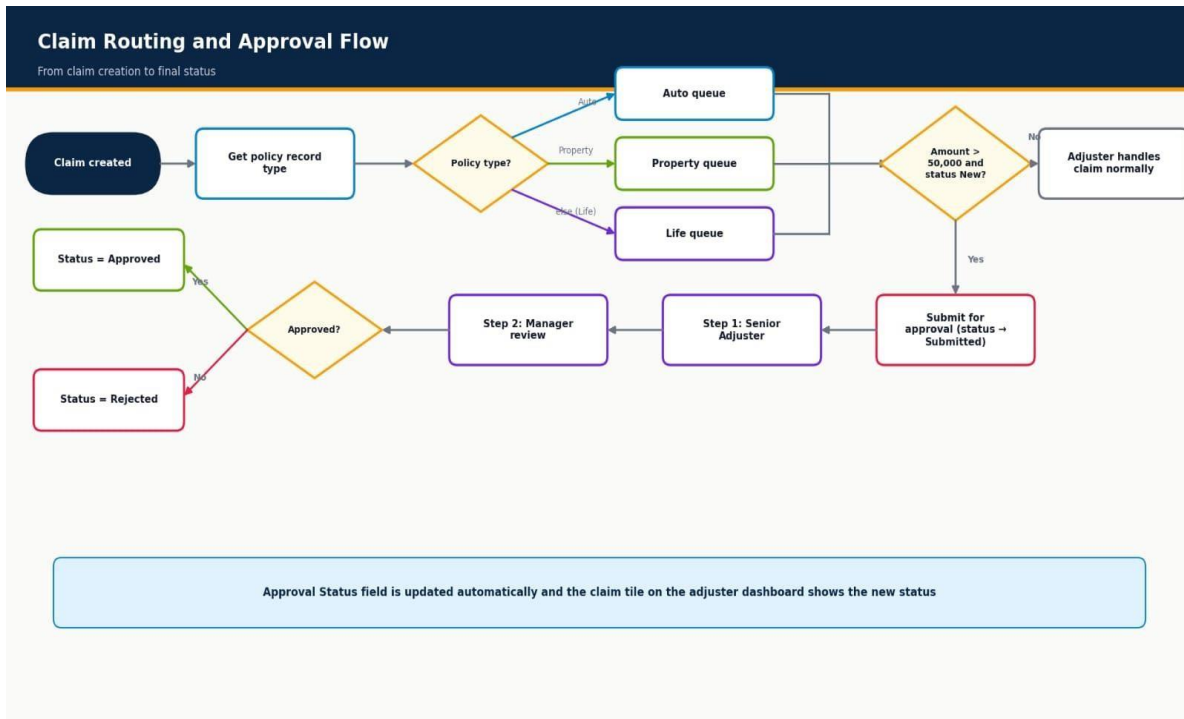
Approve; default outcome "Decision Rejected".

- On the approved path add a Submit for Approval action "Approve Claim", and on the rejected path "Reject Claim". Both use recordId as the Record ID and pass the approver comments as the submission comments.
- Save and activate as Claim Approver Screen Flow.



4.1 Create a Quick Action

- Object Manager → Claim → Buttons, Links and Actions → New Action; type Flow; choose the approver flow; label it Approve/Reject Claim; Save.
- Open the Claim page layout → Mobile & Lightning Actions and drag Approve/Reject Claim into the actions list; Save.



5. Create the Apex test class

Create ClaimsAdjusterControllerTest in Developer Console, paste the code, save, then use Run Test and check the Tests tab for coverage of the controller.

@isTest

```
private class ClaimsAdjusterControllerTest {
```

```
    @testSetup
```

```
    static void setupTestData() {
```

```
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
```

```
        User u = new User(
```

```
            FirstName = 'Test', LastName = 'User', Alias = 'tuser',
```

```
            Username = 'testuser1234343@example.com.test',
```

```
            Email = 'testuser123@example.com',
```

```
            TimeZoneSidKey = 'America/Los_Angeles',
```

```
            LocaleSidKey = 'en_US',
```

```
            EmailEncodingKey = 'UTF-8',
```

```
            LanguageLocaleKey = 'en_US',
```

```
            ProfileId = p.Id
```

```
        );
```

```
        insert u;
```

```
        Id policyRT = Schema.SObjectType.Policy__c
```

```
            .getRecordTypeInfoByDeveloperName().get('Auto').getRecordTypeId();
```

```
        Id claimRT = Schema.SObjectType.Claim__c
```

```
            .getRecordTypeInfoByDeveloperName().get('Accident').getRecordTypeId();
```

```
        Contact con = new Contact(FirstName = 'John', LastName = 'Doe', OwnerId = u.Id);
```

```
        insert con;
```

```
        Policy__c policy = new Policy__c(
```

```
            Customer__c = con.Id, RecordTypeId = policyRT,
```

```
            OwnerId = u.Id, VIN__c = '1A903284375893532'
```

```
        );
```

```
        insert policy;
```

```
        Claim__c claim = new Claim__c(
```

```
            Policy__c = policy.Id, RecordTypeId = claimRT,
```

```
            Claim_Amount__c = 5000,
```

```
            Date_of_Loss__c = Date.today().addDays(-10),
```

```
            OwnerId = u.Id
```

```
        );
```

```
        insert claim;
```

```
    }
```

```

@isTest
static void testGetAssignedClaims() {
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
    System.runAs(u) {
        Test.startTest();
        List<ClaimsAdjusterController.ClaimWrapper> results =
            ClaimsAdjusterController.getAssignedClaims();
        Test.stopTest();

        ClaimsAdjusterController.ClaimWrapper wrap = results[0];
        System.assertNotEquals(null, wrap.claimId);
        System.assertNotEquals(null, wrap.claimNumber);
        System.assertEquals('New', wrap.status);
        System.assertEquals(5000, wrap.claimAmount);
        System.assert(wrap.policyHolderName.contains('John'));
        System.assertNotEquals('Unknown', wrap.policyType);
    }
}
}

```

The screenshot displays the Visual Studio Code interface with the Test Explorer panel open. The Test Explorer shows a list of tests with their durations, results, and error messages. Below the test list, there are two panels for Code Coverage: Overall Code Coverage and Class Code Coverage.

Test	Duration	Result	Error Message
Dashboard_Pal_DashboardTest	0.01	Pass	
trthdIps_t_UtiControllerTest_getBrokerUser	0.00	Pass	
trthdIps_t_UtiControllerTest_getResetPassword	0.00	Pass	
trthdIps_t_UtiControllerTest_getCheckMyPassword	0.00	Pass	
AccountManagerTest	0.00	Pass	
AccountProcessorTest	0.00	Pass	
AddPrimaryContactTest	0.01	Pass	
AnimalLocatorTest	0.00	Pass	
ClaimsAdjusterControllerTest_getClaimable	0.00	Pass	
ClaimsAdjusterControllerTest_getAssignedClaims	0.00	Pass	
DailyLeadProcessorTest	0.02	Pass	
FileUtilitiesTest	0.00	Fail	System.DmlException: Insert failed. First e...
FileUtilitiesTest	0.01	Fail	System.DmlException: Insert failed. First e...
FileUtilitiesTest	0.00	Fail	System.DmlException: Insert failed. First e...
GeocodingServiceTest	0.00	Pass	
GeocodingServiceTest	0.00	Pass	
LeadLocatorTest	0.00	Pass	
PalLocatorTest	0.00	Pass	
testCallout	0.02	Pass	

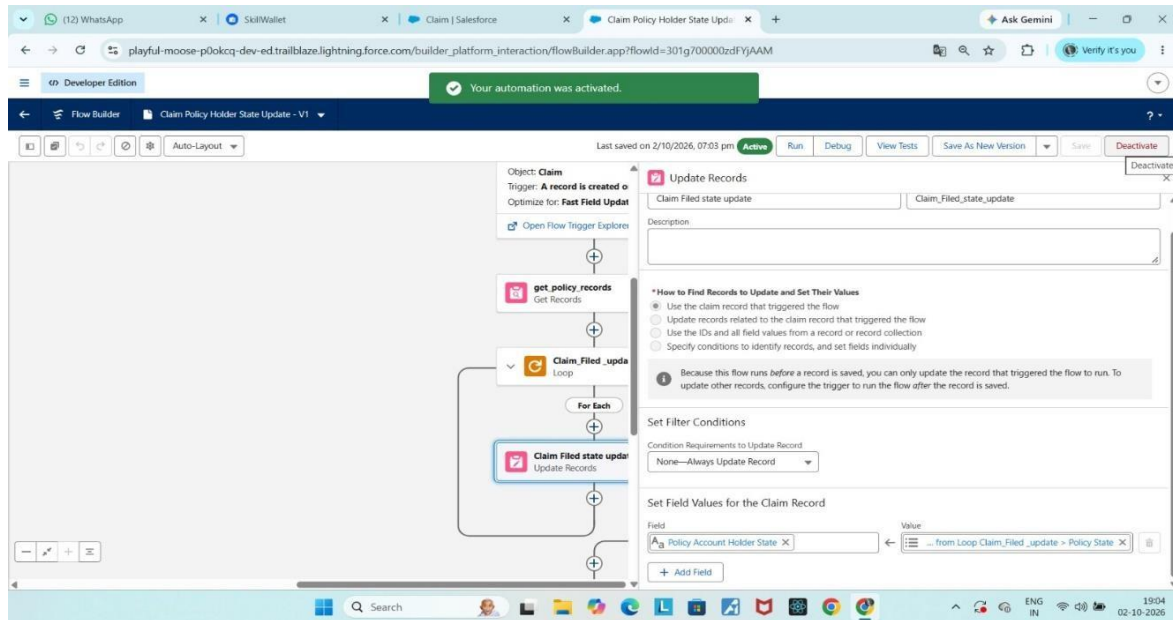
Overall Code Coverage		
Class	Percent	Lines
Overall	65%	66
LeadLocator	100%	77
AccountProcessor	100%	212
DailyLeadProcessor	100%	1212
AddPrimaryContact	100%	160
AnimalLocator	0%	0

Class Code Coverage		
Class	Percent	Lines

6. Refine the Sharing Rules (state-based access)

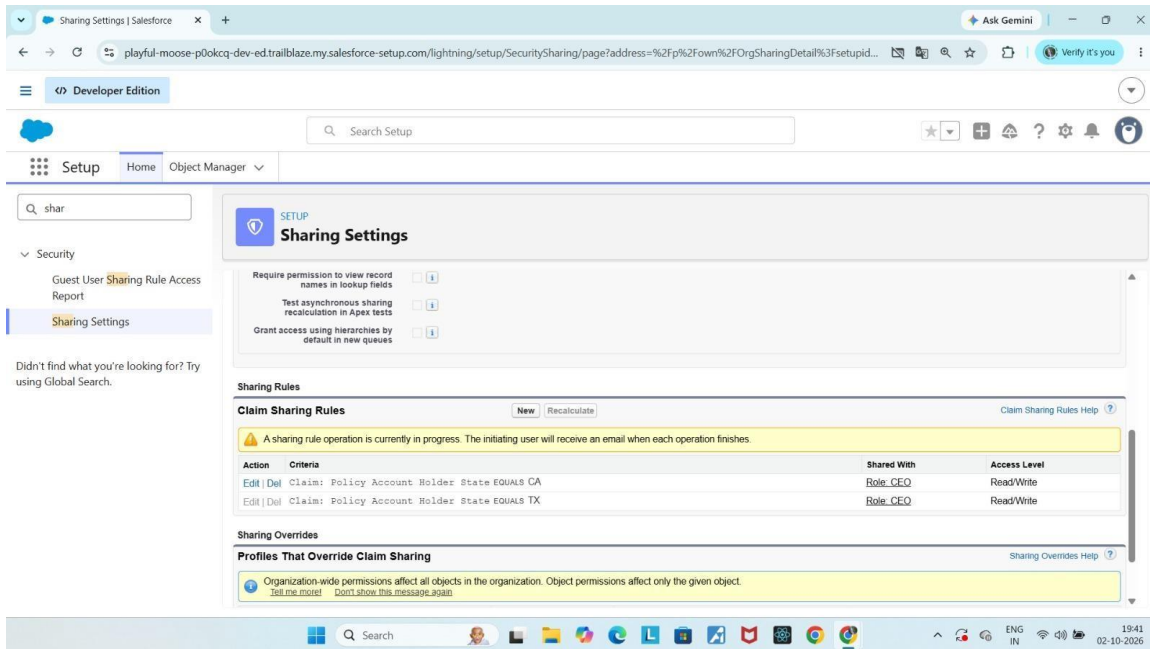
First, create a Record-Triggered Flow "Claim Policy Holder State Update" so each claim carries its policy's state:

- New Record-Triggered Flow on Claim, created or updated, no entry conditions, optimised for updates.
- Get Records "get_policy_records" (no filter, all records, store Policy State).
- Loop "Claim_Filed_update" over the policies collection, first item to last.
- Inside the loop, an Update Records element "Claim_Filed_state_update" sets the claim's Policy Account Holder State from the current loop policy state.
- Save and activate.



Then define the sharing rules (Setup → Sharing Settings → Claim → Claim Sharing Rules → New):

- Rule "Share Records based on Policy State CA": criteria Policy Account Holder State equals CA; access Read/Write.
- Rule "Share Records based on Policy State TX": criteria Policy Account Holder State equals TX; access Read/Write.



7. Create the Permission Sets

Permission Set	Access granted
Insurance Agent Access (quoting only)	Policy: Create, Read. Contact: Read, Edit. Claim: Read only. System: Flow User enabled. Apex class Premium Calculator enabled.
Claims Manager Access (reporting / approvals)	Policy: Read only. Contact: Read, Edit. Claim: Read, Edit, Delete. System: Run Reports and Manage Approvals enabled.
Claims Adjuster Access (claim handling)	Policy: Read only. Claim: Read, Edit. Apex class Claims Adjuster Controller enabled.

Browser tabs: (12) WhatsApp, SkillWallet, Permission Sets | Salesforce, Ask Gemini. Address bar: playful-moose-p00kcq-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/PermSets/home. Breadcrumbs: Setup > Home > Object Manager > Permission Sets. Search bar: Search Setup. Left sidebar: Hyperforce Assistant, Users, Permission Set Groups, Permission Sets, Apps, Mobile Apps, Salesforce, Mobile Builder for the Seller-Focused Experience, Feature Settings, Digital Experiences, Salesforce CMS, Settings, Functions. Main content: SETUP Permission Sets. On this page you can create, view, and manage permission sets. Actions: All, Edit, Delete, Create New View. Table: 1-1 of 1, 0 Selected. Table headers: Action, Permission Set Name, Description, License. Table data: Del | Clone Insurance Agent Access. Bottom bar: Windows taskbar with search, taskbar icons, system tray (ENG IN, 19:50, 02-10-2026).

Browser tabs: Permission Sets | Salesforce, Ask Gemini. Address bar: playful-moose-p00kcq-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/PermSets/home. Breadcrumbs: Setup > Home > Object Manager > Permission Sets. Search bar: Search Setup. Left sidebar: Hyperforce Assistant, Users, Permission Set Groups, Permission Sets, Apps, Mobile Apps, Salesforce, Mobile Builder for the Seller-Focused Experience, Feature Settings, Digital Experiences, Salesforce CMS, Settings, Functions. Main content: SETUP Permission Sets. On this page you can create, view, and manage permission sets. Actions: All, Edit, Delete, Create New View. Table: 1-13 of 13, 0 Selected. Table headers: Action, Permission Set Name, Description, License. Table data: Clone C360 High Scale Flow Integration User, Clone CRM User, Del | Clone Claims Adjuster Access, Del | Clone Claims Manager Access, Clone Command Center, Clone Commerce Admin, Clone ConnectivityServiceCASCPermSet, Clone Constraint Rules Engine Licenseless permission set. Bottom bar: Windows taskbar with search, taskbar icons, system tray (ENG IN, 20:14, 02-10-2026).

6. Functional and Performance Testing

Screenshots were captured from the Salesforce org to verify each feature: Policy and Claim creation, Record Types, the AutoQuoting flow, VIN validation, premium calculation, claim routing to queues, the high-value approval process, the Claims Dashboard, the Apex test results, sharing rules and Permission Sets. All features were checked and worked as expected.

7. Advantages and Disadvantages

Advantages

- **Centralised platform** – Vehicle, Property and Life insurance managed in one Salesforce system.
- **Automated quoting** – Screen Flows make quoting simple and standard.
- **Automated premium calculation** – Apex produces consistent premiums.
- **Automated claim routing** – claims reach the right queue by policy type.
- **Faster claim processing** – approvals and escalation need less manual effort.
- **Better visibility** – the LWC dashboard gives adjusters a focused view.
- **Improved data accuracy** – validation rules keep policy data correct.
- **Flexible data model** – Record Types and Field Sets support different products.
- **Secure access** – Permission Sets and Sharing Rules give role- and state-based access.
- **Scalable** – the design can add more insurance lines and features later.

Disadvantages

- **Complex data management** – the three policy types need different fields.
- **Approval complexity** – approval depends on claim amount, policy type and adjuster level.
- **User adoption** – agents need intuitive flows to keep clicks low.
- **Salesforce dependency** – the solution relies on Salesforce configuration and platform capabilities.
- **Implementation effort** – many Flows, Apex classes, LWC components, rules and approvals need careful building and upkeep.

8. Conclusion

The Multi-Line Insurance Policy and Claims Management System gives insurance teams one Salesforce solution for Vehicle, Property and Life policies and claims. It removes manual and fragmented working by adding guided quoting, automatic premium calculation, claim routing, structured approvals and an LWC Claims Dashboard.

By combining Flows, Apex, Record Types, Field Sets, Validation Rules, Approval Processes, LWC, Permission Sets and Sharing Rules, the project improves policy processing, claim handling, data accuracy, security and visibility. It shows how Salesforce can unite several insurance processes on one scalable platform and serve customers better.

9. Future Scope

- **External integration** – connect to vehicle and property databases for real-time quote verification.
- **More insurance products** – extend beyond Vehicle, Property and Life.
- **Advanced claims automation** – automate claim assessment and escalation using claim characteristics.
- **Better analytics** – richer dashboards and reports on policies, premiums and claims.
- **Customer experience** – a customer-facing view for policy and claim status.
- **Mobile access** – let agents and claims users run key processes from mobile devices.
- **AI assistance** – explore AI help for agents and adjusters.
- **API integrations** – widen API-based links with external insurance and verification systems.

10. Appendix

- **Objects:** Policy, Claim, Contact, User
- **Record Types:** Vehicle (Auto), Property, Life; Accident, Property, Life
- **Automation:** Screen Flows, Record-Triggered Flows, Approval Processes

- **Apex:** PremiumCalculator, ClaimsAdjusterController
- **LWC:** claimsDashboardLwc, claimTileLwc
- **Validation:** 17-character VIN rule
- **Security:** Permission Sets, state-based Sharing Rules
- **Testing:** ClaimsAdjusterControllerTest
- **Reporting:** Claims Dashboard and Salesforce Reports / Dashboards